Solucionario razonado

Series de Taylor y termodinámica del sistema Fe-Al-O

Edición desarrollada paso a paso

Primer parcial de Matemáticas Aplicadas · Incisos a–g

Para quién está escrito. Para una persona que necesita ver de dónde sale cada símbolo, qué se toma del artículo, qué se calcula y cómo se llega de una línea a la siguiente. No se presupone que haya leído el artículo ni que sepa programar.

Qué se entrega. Explicación física, procedencia de los parámetros, desarrollo algebraico de la energía libre, derivación de las propiedades parciales molares, derivadas de Taylor una por una, sustituciones numéricas, integración de Gibbs-Duhem, cálculo del error, construcción de la serie infinita y programas completos en Python y MATLAB, comentados instrucción por instrucción.

Qué se compara. Aproximaciones de un modelo publicado. Los puntos calculados no son mediciones experimentales. La referencia cúbica no es la función exacta desconocida de la naturaleza.

Cómo leerlo. Seguir las secciones en orden la primera vez. Las cuentas manuales se muestran con ejemplos completos; las operaciones repetidas sobre la malla de 10 001 puntos se registran en archivos CSV, además de explicarse mediante fórmulas e instrucciones. Una serie infinita no puede escribirse término por término hasta el infinito: aquí se deriva la regla que determina cualquier coeficiente y se desglosan todos los utilizados, de grado 0 a 12.

Archivos y descarga

`https://github.com/Sekinal/rey`

Fuente editable: `documentos/solucionario.tex`.

Python: `ejemplos/paso_a_paso.py`.

MATLAB: `matlab/paso_a_paso.m`.

Generación integral: `uv run --locked feal`.

Edición del 10 de octubre de 2026.

Índice de lectura

1 Antes de empezar: símbolos y operaciones básicas 3

2 a) El sistema físico: qué sucede en el metal 4

3 a) De tres componentes a dos: hipótesis y unidades 5

4 Mapa de procedencia: de dónde sale cada número 6

5 b) Por qué Taylor y qué función vamos a desarrollar 7

6 c) Expandir la energía libre: cada multiplicación 8

7 c) Derivar la energía libre término por término 9

8 c) De la energía molar a la contribución de cada especie 10

9 c) Construir el polinomio del aluminio, sin omitir productos 11

10 c) Taylor desde su definición: por qué aparecen factoriales 12

11 c) Taylor del modelo: todas las derivadas y sus valores 13

12 c) El redondeo del artículo y el resto de Taylor 14

13 d) Construir cada aproximación y hacer una cuenta completa 15

14 d) Del ejemplo a todos los puntos de cálculo 16

15 d) Error absoluto: restar primero, tomar valor absoluto después 17

16 d) Error del coeficiente: por qué aparece otra exponencial 18

17 d) Métricas globales: máximo y RMSE, operación por operación 19

18 d) Por qué agregar un término puede empeorar el máximo 20

19 d) Tolerancia de 1 por ciento: resolver la desigualdad 21

20 e) Leer las gráficas: qué representa cada eje 22

21 c) Por qué el hierro necesita su propia derivación 23

22 c) Integrar Gibbs-Duhem: resolver las tres integrales 24

23 c) Hierro de cada orden y cancelación del logaritmo 25

24 c) Fórmula general y precisión cerca de cero **26**

25 f) Energía de exceso: reconstruirla desde Al y Fe **27**

26 f) Cuenta completa en el extremo de mayor composición **28**

27 Extensión: distinguir dos aproximaciones que no son iguales **29**

28 Extensión: derivar la recurrencia sin saltos de índices **30**

29 Extensión: calcular a mano los primeros coeficientes **31**

30 Extensión: desglose numérico de todos los coeficientes utilizados **32**

31 Extensión: construir y evaluar las aproximaciones infinitas truncadas **36**

32 f) Interpretar los resultados físicos con sus signos **37**

33 f) Energía negativa, calor y estabilidad: tres conceptos distintos **38**

34 g) Conclusiones, límites y cómo justificar cada afirmación **39**

35 e) El código: qué archivo abrir y cómo ejecutarlo **40**

36 e) Leer Python desde cero: signos, arreglos e índices **41**

37 e) Seguir un ciclo a mano y entender las funciones matemáticas **42**

38 e) Del cálculo a los archivos y a cada gráfica **43**

39 e) Código didáctico completo, con comentarios instrucción por instrucción **44**

40 Versión MATLAB: archivo completo y forma de ejecución **49**

41 MATLAB desde cero: las diferencias que pueden cambiar una cuenta **50**

42 MATLAB: seguir las cuentas y comprobar la equivalencia **51**

43 Código MATLAB completo, comentado instrucción por instrucción **52**

44 Cómo funciona la reproducción editorial y cómo se comprueba **57**

45 Guía para responder las dudas que motivaron esta edición **58**

46 Fuentes y correspondencia con el examen **59**

1 Antes de empezar: símbolos y operaciones básicas

1. Una función es una regla de cálculo.

Si escribimos $F(x)$, x es la entrada y $F(x)$ la salida. Por ejemplo, si $F(x) = 2 + 3x$, entonces $F(0.1) = 2 + 3(0.1) = 2 + 0.3 = 2.3$. Los paréntesis después del nombre indican evaluación; no significan que se multiplique F por x.

2. Potencias y signos.

$x^2 = x \cdot x$, $x^3 = x \cdot x \cdot x$ y $x^0 = 1$, también en la evaluación del término constante de un polinomio en cero. Para $x = 0.1$: $x^2 = 0.01$ y $x^3 = 0.001$. Multiplicar dos negativos da positivo: $(-2)(-4.6095) = 9.219$. Multiplicar un negativo por un positivo da negativo: $(-6)(0.741) = -4.446$.

3. Distribuir y agrupar.

$a(b + c) = ab + ac$. Por ejemplo, $x(1 - x) = x - x^2$. Solo se suman directamente términos de igual potencia: $2x^2 + 3x^2 = 5x^2$, pero $2x + 3x^2$ conserva dos términos.

4. Logaritmo natural y exponencial.

ln significa logaritmo natural, de base $e \simeq 2.71828$. Las operaciones ln y exp se deshacen mutuamente: si $F = \ln \gamma$, entonces $\gamma = \exp(F) = e^F$. Ejemplo: si $F = -2$, $\gamma = e^{-2} \simeq 0.135335$. Un logaritmo negativo corresponde a un coeficiente positivo menor que uno; el coeficiente no es negativo.

5. Factorial y sumatoria.

$0! = 1$, $1! = 1$, $2! = 2 \cdot 1 = 2$, $3! = 3 \cdot 2 \cdot 1 = 6$, $4! = 24$. La expresión $\sum_{k=0}^{3} c_k x^k$ significa escribir y sumar exactamente cuatro términos:

$$c_0 x^0 + c_1 x^1 + c_2 x^2 + c_3 x^3 = c_0 + c_1 x + c_2 x^2 + c_3 x^3.$$

k es un contador. c_k es el coeficiente asociado a la potencia x^k.

6. Derivada, integral y aproximación.

$F'(x)$ es la derivada: mide cuánto cambia F por un cambio pequeño de x. F'' es la derivada de F'. Una integral definida suma contribuciones acumuladas entre dos límites. Las reglas necesarias se explican antes de aplicarlas. El signo $=$ indica igualdad; $\simeq$ indica un valor aproximado o redondeado.

7. Nombres que se usarán.

$f = g^E/(RT)$ es la energía de exceso dividida entre RT; $F = \ln \gamma_{\mathrm{Al}}$ es el logaritmo del coeficiente de Al; $H = \ln \gamma_{\mathrm{Fe}}$ es el del Fe. Son funciones diferentes. P_N aproxima F reteniendo hasta grado N; Q_m aproxima directamente γ hasta grado m.

2 a) El sistema físico: qué sucede en el metal

1. Materiales presentes.

El hierro líquido es el medio principal. Contiene aluminio y oxígeno disueltos. Los corchetes [Al] y [O] señalan especies disueltas, no necesariamente sustancias puras separadas. Al agregar aluminio puede formarse alúmina sólida:

$$2[\mathrm{Al}] + 3[\mathrm{O}] \longrightarrow \mathrm{Al_2O_3}(s).$$

El 2 cuenta átomos de Al y el 3 cuenta átomos de O por unidad de alúmina. (s) significa sólido. Disminuye el oxígeno disuelto, pero pueden quedar partículas de alúmina que requieren separación.

2. Lo que estudia el artículo.

Estudia equilibrio termodinámico y no idealidad: cómo se relacionan composición, actividad y energía de Gibbs. No calcula cuánto tarda la reacción, cómo se agita el metal o a qué velocidad se retiran las partículas.

3. Fracciones molares.

Un mol es una cantidad de sustancia. Si hay n_{Al} moles de Al y n_{Fe} de Fe en el modelo binario, el total es $n = n_{\mathrm{Al}} + n_{\mathrm{Fe}}$ y

$$x_{\mathrm{Al}} = \frac{n_{\mathrm{Al}}}{n}, \quad x_{\mathrm{Fe}} = \frac{n_{\mathrm{Fe}}}{n}, \quad x_{\mathrm{Al}} + x_{\mathrm{Fe}} = \frac{n_{\mathrm{Al}} + n_{\mathrm{Fe}}}{n} = 1.$$

Definimos $x = x_{\mathrm{Al}}$; por tanto $x_{\mathrm{Fe}} = 1 - x$. Ejemplo: 1 mol de Al más 9 moles de Fe da $n = 10$, $x = 1/10 = 0.1$ y $x_{\mathrm{Fe}} = 9/10 = 0.9$. Son fracciones de moles, no de masa.

4. Actividad y coeficiente de actividad.

La actividad describe la participación termodinámica de una especie. En la escala utilizada, $a_i = x_i\gamma_i$. Si fuera ideal, $\gamma_i = 1$ y $a_i = x_i$. Si $x_i = 0.1$ y $\gamma_i = 0.05$, entonces $a_i = 0.1(0.05) = 0.005$. Comparar actividades exige conservar el mismo estado de referencia.

5. Relación con la desoxidación.

Para alúmina pura, su actividad es 1. El artículo define la constante de disociación $K_d = a_{\mathrm{Al}}^2 a_{\mathrm{O}}^3$. Dividiendo ambos lados entre a_{Al}^2:

$$\frac{K_d}{a_{\mathrm{Al}}^2} = a_{\mathrm{O}}^3.$$

Tomando raíz cúbica: $a_{\mathrm{O}} = (K_d/a_{\mathrm{Al}}^2)^{1/3}$. A T fija, K_d es fijo: aumentar a_{Al} reduce a_{O}. Para obtener $x_{\mathrm{O}} = a_{\mathrm{O}}/\gamma_{\mathrm{O}}$ hace falta también γ_{O}. El cálculo binario de este trabajo no proporciona ese equilibrio completo.

3 a) De tres componentes a dos: hipótesis y unidades

1. Qué se simplifica.

El sistema completo Fe-Al-O es ternario. El artículo propone que, cuando la contribución de O es pequeña, Fe y Al pueden describirse aproximadamente como un binario. Esto es una hipótesis física, anterior a cualquier Taylor. El artículo sitúa esa reducción por encima de aproximadamente 0.07 % en masa de Al en las condiciones estudiadas [1, sección 3.2].

2. Convertir 0.07 por ciento en masa a fracción molar.

"Por ciento" significa por cada 100. Tomemos 100 g de mezcla binaria:

$$m_{\mathrm{Al}} = 0.07 \text{ g}, \qquad m_{\mathrm{Fe}} = 100 - 0.07 = 99.93 \text{ g}.$$

Con masas molares $M_{\mathrm{Al}} = 26.98$ g/mol y $M_{\mathrm{Fe}} = 55.85$ g/mol, la regla $n = m/M$ da

$$n_{\mathrm{Al}} = \frac{0.07}{26.98} \simeq 0.002594514 \text{ mol}, \qquad n_{\mathrm{Fe}} = \frac{99.93}{55.85} \simeq 1.789256938 \text{ mol}.$$

Sumamos: $n \simeq 0.002594514 + 1.789256938 = 1.791851452$ mol. Dividimos la cantidad de Al entre el total:

$$x_{\mathrm{Al}} \simeq \frac{0.002594514}{1.791851452} \simeq 0.001447952.$$

Así se obtiene $x \simeq 0.00145$, que no es ni 0.07 ni 0.0007. Este cálculo supone solo Fe y Al; no es una conversión exacta de toda composición ternaria.

3. Temperatura y energía.

Usamos $T = 1873$ K. En grados Celsius, $1873 - 273.15 = 1599.85\,^{\circ}\mathrm{C}$. La constante es $R = 8.314462618$ J/(mol K). Su producto es

$$RT = (8.314462618)(1873) = 15572.986483514 \text{ J/mol}.$$

Los kelvin se cancelan: [J/(mol K)][K] = J/mol. Por eso $g^E/(RT)$ no tiene unidades cuando g^E está en J/mol. 1 kJ = 1000 J: para convertir J/mol a kJ/mol dividimos entre 1000.

4. Qué es energía de exceso.

g^E es la diferencia entre la energía de Gibbs molar de la solución real modelada y la de una solución ideal de igual composición y mismos estados de referencia. "Molar" significa por mol. $g^E < 0$ no significa, por sí solo, que la mezcla libere esa cantidad de calor.

4 Mapa de procedencia: de dónde sale cada número

Dato	Valor o expresión	Procedencia y uso
A	-4.6095	Tabla 1 de [1], coeficiente binario Fe-Al de Redlich-Kister.
B	0.741	Tabla 1 de [1], segundo coeficiente binario Fe-Al.
Modelo f	$x(1-x)[A+B(1-2x)]$	Ecuación (16) de [1]. Se toma como punto de partida; no se ajusta aquí a mediciones.
T	1873 K	Condición de estudio de [1].
R	8.314462618 J/(mol K)	Constante física adoptada para convertir energía adimensional a J/mol.
c_0, c_1, c_2, c_3 derivados	-3.8685, 4.773, 2.0595, -2.964	Se obtienen de A, B mediante la derivación que se muestra a continuación.
c_2 publicado	2.06	Valor redondeado usado en la ecuación (26) de [1]. Se declara el cambio explícitamente.
$x_{\max}$	0.3872	Última composición de Al de la tabla 2 de [1].
M	10 001	Elección numérica de este solucionario; no son 10 001 experimentos.
$0.01, 0.1, 0.2, 0.3$	Composiciones ilustrativas	Elegidas aquí para mostrar cuentas; no se presentan como datos experimentales.
$b_0, \ldots, b_{12}$	Coeficientes de e^{F_3}	Se calculan por una recurrencia derivada aquí. No proceden de otra tabla del artículo.

La cadena completa.

Artículo $\to A, B, f \to$ derivación de $F \to$ Taylor $\to P_N \to \gamma_N = e^{P_N} \to$ errores. En otra rama: $P_N \to$ Gibbs-Duhem $\to H_N \to g_N^E$. Para la extensión infinita: $F_3 \to e^{F_3} \to b_n \to Q_m$.

No se digitalizan puntos de una imagen ni se inventa una tabla experimental. Se generan pares (x, valor calculado) evaluando ecuaciones. La tabla 2 sirve como referencia de escala, pero sus últimas cifras no se imponen como si fueran datos exactos de este programa.

5 b) Por qué Taylor y qué función vamos a desarrollar

1. La pregunta matemática.

Queremos conocer cómo cambia $\ln \gamma_{\mathrm{Al}}$ cuando x crece desde una cantidad muy pequeña de Al. El punto de expansión es $x_0 = 0$. "Alrededor de cero" significa construir las aproximaciones usando el valor y las derivadas evaluados en cero. No significa sustituir $x = 0$ en todas las gráficas: después evaluaremos el polinomio en muchos valores de x.

2. Por qué se usa el logaritmo.

La termodinámica relaciona el potencial químico con la actividad mediante $\mu_i = \mu_i^\circ + RT \ln a_i$. Como $a_i = x_i \gamma_i$, la propiedad $\ln(ab) = \ln a + \ln b$ da

$$\mu_i = \mu_i^\circ + RT \ln x_i + RT \ln \gamma_i.$$

Los primeros dos términos constituyen la contribución ideal. La diferencia respecto de ella es $\mu_i^E = RT \ln \gamma_i$. De aquí sale la relación entre energía de exceso y logaritmo del coeficiente; no es una elección arbitraria de notación.

3. Qué significa dilución infinita.

Es el límite de una cantidad de soluto tan pequeña que $x \to 0$. El modelo asigna un valor límite finito a γ_{Al}, aunque en hierro exactamente puro no haya una cantidad finita de Al cuya actividad medir. Se usa ese límite para definir las derivadas del modelo. Despreciar O en esta zona sigue siendo una hipótesis que puede fallar físicamente.

4. Por qué no Fourier.

Fourier resulta natural para comportamientos periódicos. Aquí la variable es composición, no un tiempo o posición con repetición periódica. El modelo es suave en composición y queremos organizarlo en potencias de la distancia a cero: esa es la estructura de Taylor.

5. La advertencia clave.

El modelo de partida ya es polinómico. Primero derivaremos de la energía de exceso un polinomio cúbico para F. Después aplicaremos Taylor explícitamente a ese polinomio. El Taylor de grado tres devolverá el mismo polinomio. La comparación interesante será observar qué se pierde al cortar antes. En una extensión separada, e^{F_3} sí tendrá una serie infinita.

Qué acabamos de hacer. Todavía no hemos calculado un coeficiente de Taylor. Hemos definido la función de interés, el punto de expansión y la razón física para usar $\ln \gamma$.

6 c) Expandir la energía libre: cada multiplicación

1. Copiar el modelo y definir un nombre corto.

La ecuación (16) de [1] da

$$f(x) = \frac{g^E(x)}{RT} = x(1-x)[A + B(1-2x)].$$

f es adimensional. A y B son constantes respecto de x a la temperatura fijada.

2. Abrir el primer paréntesis.

Multiplicamos x por ambos términos: $x(1-x) = x \cdot 1 - x \cdot x = x - x^2$.

3. Abrir el segundo paréntesis.

$A + B(1-2x) = A + B \cdot 1 - B \cdot 2x = A + B - 2Bx$. Por tanto,

$$f = (x - x^2)(A + B - 2Bx).$$

4. Distribuir los dos términos del primer factor.

$$\begin{aligned} f &= x(A + B - 2Bx) - x^2(A + B - 2Bx) \\ &= Ax + Bx - 2Bx^2 - Ax^2 - Bx^2 + 2Bx^3. \end{aligned}$$

El último signo es positivo porque $(-x^2)(-2Bx) = +2Bx^3$.

5. Agrupar por potencia.

$$\begin{aligned} &\text{Términos en } x : Ax + Bx = (A + B)x, \\ &\text{Términos en } x^2 : -2Bx^2 - Ax^2 - Bx^2 = (-A - 3B)x^2, \\ &\text{Término en } x^3 : 2Bx^3. \end{aligned}$$

Así,

$$f = (A + B)x - (A + 3B)x^2 + 2Bx^3.$$

6. Sustituir las constantes con sus signos.

$$\begin{aligned} A + B &= -4.6095 + 0.741 = -3.8685, \\ A + 3B &= -4.6095 + 3(0.741) = -4.6095 + 2.223 = -2.3865, \\ -(A + 3B) &= -(-2.3865) = +2.3865, \\ 2B &= 2(0.741) = 1.482. \end{aligned}$$

Por tanto $f(x) = -3.8685x + 2.3865x^2 + 1.482x^3$. Esto sigue siendo energía de exceso dividida entre RT; todavía no es $\ln \gamma_{\mathrm{Al}}$.

7 c) Derivar la energía libre término por término

1. Regla de la potencia.

Si a es constante, $d(ax^m)/dx = max^{m-1}$. Se multiplica por el exponente y luego se resta uno al exponente. Ejemplo: $d(7x^3)/dx = 3(7)x^2 = 21x^2$. La derivada de una constante es cero.

2. Aplicar la regla a cada sumando de f.

$$\begin{aligned}\frac{d}{dx}[(A+B)x] &= (A+B)(1)x^0 = A+B,\\ \frac{d}{dx}[-(A+3B)x^2] &= -2(A+3B)x,\\ \frac{d}{dx}[2Bx^3] &= 3(2B)x^2 = 6Bx^2.\end{aligned}$$

La derivada de una suma es la suma de las derivadas:

$$f'(x) = (A+B) - 2(A+3B)x + 6Bx^2.$$

3. Calcular los coeficientes numéricos.

$$\begin{aligned}A+B &= -3.8685,\\ -2(A+3B) &= -2(-2.3865) = 4.773,\\ 6B &= 6(0.741) = 4.446.\end{aligned}$$

Entonces $f'(x) = -3.8685 + 4.773x + 4.446x^2$.

4. Comprobar con la forma numérica.

Partiendo de $f = -3.8685x + 2.3865x^2 + 1.482x^3$, derivamos directamente:

$$f' = -3.8685 + (2)(2.3865)x + (3)(1.482)x^2 = -3.8685 + 4.773x + 4.446x^2.$$

Ambas rutas coinciden.

5. Por qué necesitamos otra derivada distinta.

f' mide el cambio de la energía molar adimensional con la composición. Pero $\ln\gamma_i$ está relacionado con el cambio de la energía *total* cuando añadimos moles de una especie manteniendo fija la otra. Por eso la siguiente sección usa derivadas respecto de n_{Al} y n_{Fe}, no simplemente f'.

Qué acabamos de hacer. Ya podemos derivar un polinomio. Falta convertir una propiedad molar de la mezcla en la contribución parcial molar de cada especie.

8 c) De la energía molar a la contribución de cada especie

1. Energía total.

Si cada mol tiene energía de exceso $g^E = RTf$, entonces n moles tienen $G^E = ng^E = nRTf(x)$. G^E se mide en J y g^E en J/mol. La contribución parcial molar es

$$\mu_i^E = \left(\frac{\partial G^E}{\partial n_i}\right)_{T,P,n_{j\neq i}}.$$

El símbolo ∂ indica que una variable cambia y otras se mantienen fijas. Usamos la identidad termodinámica $\mu_i^E = RT\ln\gamma_i$ explicada antes.

2. Calcular cómo cambia x al añadir Al.

Tenemos $x = n_{\mathrm{Al}}/(n_{\mathrm{Al}} + n_{\mathrm{Fe}})$. La regla del cociente es $(u/v)' = (u'v - uv')/v^2$. Manteniendo n_{Fe} fijo, $u' = 1$ y $v' = 1$:

$$\begin{aligned}\frac{\partial x}{\partial n_{\mathrm{Al}}} &= \frac{1(n_{\mathrm{Al}} + n_{\mathrm{Fe}}) - n_{\mathrm{Al}}(1)}{(n_{\mathrm{Al}} + n_{\mathrm{Fe}})^2} = \frac{n_{\mathrm{Fe}}}{n^2} \\ &= \frac{n_{\mathrm{Fe}}/n}{n} = \frac{1-x}{n}.\end{aligned}$$

3. Calcular cómo cambia x al añadir Fe.

Ahora n_{Al} permanece fijo: $u' = 0$ y $v' = 1$:

$$\frac{\partial x}{\partial n_{\mathrm{Fe}}} = \frac{0(n_{\mathrm{Al}} + n_{\mathrm{Fe}}) - n_{\mathrm{Al}}(1)}{n^2} = -\frac{n_{\mathrm{Al}}}{n^2} = -\frac{x}{n}.$$

4. Derivar $G^E = nRTf(x)$ respecto de Al.

RT es constante. Para un producto, $(uv)' = u'v + uv'$. Para una función compuesta, $\partial f(x)/\partial n_i = f'(x)\partial x/\partial n_i$. Además, $\partial n/\partial n_i = 1$. Juntando las reglas:

$$\begin{aligned}\frac{\partial G^E}{\partial n_{\mathrm{Al}}} &= RT\left[\frac{\partial n}{\partial n_{\mathrm{Al}}}f + nf'\frac{\partial x}{\partial n_{\mathrm{Al}}}\right] \\ &= RT\left[1\cdot f + nf'\frac{1-x}{n}\right] = RT[f + (1-x)f'].\end{aligned}$$

Igualamos a $RT\ln\gamma_{\mathrm{Al}}$ y dividimos entre RT:

$$\boxed{\ln\gamma_{\mathrm{Al}} = f + (1-x)f'.}$$

5. Repetir para Fe.

$RT\ln\gamma_{\mathrm{Fe}} = RT[1\cdot f + nf'(-x/n)] = RT[f - xf']$. Dividiendo entre RT:

$$\boxed{\ln\gamma_{\mathrm{Fe}} = f - xf'.}$$

9 c) Construir el polinomio del aluminio, sin omitir productos

1. Escribir $F = f + (1-x)f' = f + f' - xf'$.

Ya sabemos que $f = (A+B)x - (A+3B)x^2 + 2Bx^3$ y $f' = (A+B) - 2(A+3B)x + 6Bx^2$.

2. Multiplicar f' por $-x$.

$$-xf' = -(A+B)x + 2(A+3B)x^2 - 6Bx^3.$$

Cada término de f' se multiplica por $-x$; el término en x pasa a x^2, y el de x^2 a x^3.

3. Escribir los tres sumandos completos.

$$\begin{aligned} F = &\underbrace{(A+B)x - (A+3B)x^2 + 2Bx^3}_{f} \\ &+ \underbrace{(A+B) - 2(A+3B)x + 6Bx^2}_{f'} \\ &+ \underbrace{[-(A+B)x + 2(A+3B)x^2 - 6Bx^3]}_{-xf'}. \end{aligned}$$

4. Agrupar una potencia a la vez.

$$\begin{aligned} x^0: &\quad (A+B), \\ x^1: &\quad (A+B) - 2(A+3B) - (A+B) \\ &\quad = -2(A+3B) = -2A - 6B, \\ x^2: &\quad -(A+3B) + 6B + 2(A+3B) \\ &\quad = -A - 3B + 6B + 2A + 6B = A + 9B, \\ x^3: &\quad 2B - 6B = -4B. \end{aligned}$$

5. Sustituir cada número.

$$\begin{aligned} A + B &= -4.6095 + 0.741 = -3.8685, \\ -2A - 6B &= -2(-4.6095) - 6(0.741) = 9.219 - 4.446 = 4.773, \\ A + 9B &= -4.6095 + 9(0.741) = -4.6095 + 6.669 = 2.0595, \\ -4B &= -4(0.741) = -2.964. \end{aligned}$$

Con el subíndice RK para identificar su procedencia:

$$\boxed{F_{\mathrm{RK}}(x) = -3.8685 + 4.773x + 2.0595x^2 - 2.964x^3.}$$

Qué acabamos de hacer. Este polinomio salió de la energía de exceso mediante propiedades parciales molares. Todavía no hemos aplicado Taylor. La sección siguiente sí lo hace explícitamente.

10 c) Taylor desde su definición: por qué aparecen factoriales

1. Proponer un polinomio que imite a F cerca de cero.

Escribimos $P_3(x) = c_0 + c_1x + c_2x^2 + c_3x^3$. Queremos que coincidan el valor y las tres primeras derivadas de P_3 y F en cero. Así ambos tienen allí el mismo valor, pendiente y primeras correcciones de curvatura.

2. Igualar el valor en cero.

$P_3(0) = c_0 + c_1(0) + c_2(0)^2 + c_3(0)^3 = c_0$. Como exigimos $P_3(0) = F(0)$, resulta $c_0 = F(0)$.

3. Igualar la primera derivada.

$P_3' = c_1 + 2c_2x + 3c_3x^2$. Al evaluar en cero, $P_3'(0) = c_1$. Exigir $P_3'(0) = F'(0)$ da $c_1 = F'(0)$.

4. Igualar la segunda derivada.

Derivamos otra vez: $P_3'' = 2c_2 + 6c_3x$. Entonces $P_3''(0) = 2c_2$. Igualamos $2c_2 = F''(0)$ y dividimos entre 2:

$$c_2 = \frac{F''(0)}{2} = \frac{F''(0)}{2!}.$$

5. Igualar la tercera derivada.

$P_3''' = 6c_3$, de modo que $6c_3 = F'''(0)$ y $c_3 = F'''(0)/6 = F'''(0)/3!$.

6. Escribir la regla general.

Al derivar x^k exactamente k veces aparecen los factores $k(k-1)\cdots 1 = k!$. Por eso el coeficiente es $c_k = F^{(k)}(0)/k!$, y no solo la derivada. Taylor con resto escribe

$$F(x) = \underbrace{\sum_{k=0}^{N} \frac{F^{(k)}(0)}{k!}x^k}_{P_N(x)} + R_N(x).$$

$R_N = F - P_N$ es lo que falta. Para un centro general x_0 usaríamos $(x - x_0)^k$ y derivadas en x_0. Aquí $x_0 = 0$ y $x - 0 = x$; este caso también se llama serie de Maclaurin.

Qué acabamos de hacer. Los factoriales compensan las multiplicaciones que produce la derivación repetida. No son números agregados sin justificación. Ahora aplicaremos esta construcción a F_{RK}.

11 c) Taylor del modelo: todas las derivadas y sus valores

Partimos de $F_{\mathrm{RK}} = -3.8685 + 4.773x + 2.0595x^2 - 2.964x^3$.

1. Primera derivada: un término por línea.

$$\begin{aligned}(-3.8685)' &= 0, \qquad (4.773x)' = 4.773(1)x^0 = 4.773,\\ (2.0595x^2)' &= 2(2.0595)x = 4.119x,\\ (-2.964x^3)' &= 3(-2.964)x^2 = -8.892x^2.\end{aligned}$$

Sumamos: $F'_{\mathrm{RK}} = 4.773 + 4.119x - 8.892x^2$.

2. Segunda derivada.

$(4.773)' = 0$, $(4.119x)' = 4.119$, $(-8.892x^2)' = 2(-8.892)x = -17.784x$. Por tanto $F''_{\mathrm{RK}} = 4.119 - 17.784x$.

3. Tercera y cuarta derivadas.

$(4.119)' = 0$ y $(-17.784x)' = -17.784$, así que $F'''_{\mathrm{RK}} = -17.784$. Como es constante, $F^{(4)}_{\mathrm{RK}} = 0$. Derivar cero vuelve a dar cero: todas las derivadas posteriores son cero.

4. Evaluar en cero sin omitir los términos que desaparecen.

$$\begin{aligned}F_{\mathrm{RK}}(0) &= -3.8685 + 4.773(0) + 2.0595(0)^2 - 2.964(0)^3 = -3.8685,\\ F'_{\mathrm{RK}}(0) &= 4.773 + 4.119(0) - 8.892(0)^2 = 4.773,\\ F''_{\mathrm{RK}}(0) &= 4.119 - 17.784(0) = 4.119,\\ F'''_{\mathrm{RK}}(0) &= -17.784.\end{aligned}$$

5. Dividir cada derivada entre su factorial.

$$c_0 = \frac{-3.8685}{1} = -3.8685, \qquad c_1 = \frac{4.773}{1} = 4.773,$$
$$c_2 = \frac{4.119}{2} = 2.0595, \qquad c_3 = \frac{-17.784}{6} = -2.964.$$

6. Sustituir en la fórmula de Taylor.

$$\begin{aligned}F_{\mathrm{RK}}(x) &= F_{\mathrm{RK}}(0) + F'_{\mathrm{RK}}(0)x + \frac{F''_{\mathrm{RK}}(0)}{2!}x^2 + \frac{F'''_{\mathrm{RK}}(0)}{3!}x^3\\ &= -3.8685 + 4.773x + \frac{4.119}{2}x^2 + \frac{-17.784}{6}x^3\\ &= -3.8685 + 4.773x + 2.0595x^2 - 2.964x^3.\end{aligned}$$

Recuperamos el polinomio de partida porque es cúbico. No hubo ajuste ni descubrimiento de nuevos datos: se verificó su expansión de Taylor y se identificó qué significa cada coeficiente.

12 c) El redondeo del artículo y el resto de Taylor

1. Dos referencias cercanas, pero distintas.

La derivación desde A, B da $c_2 = 2.0595$. La ecuación (26) de [1] usa $\rho = 2.06$. Para reproducir sus números definimos explícitamente otra función:

$$F_3(x) = -3.8685 + 4.773x + 2.06x^2 - 2.964x^3.$$

Restamos término a término:

$$\begin{aligned} F_3 - F_{\mathrm{RK}} &= (-3.8685 + 3.8685) + (4.773 - 4.773)x \\ &\quad + (2.06 - 2.0595)x^2 + (-2.964 + 2.964)x^3 \\ &= 0 + 0x + 0.0005x^2 + 0x^3 = 0.0005x^2. \end{aligned}$$

En el extremo, $0.3872^2 = 0.14992384$ y $0.0005(0.14992384) = 0.00007496192$. Esta diferencia es de redondeo de parámetros, no de truncamiento.

2. Taylor de la referencia redondeada.

Derivando igual que antes:

$$F_3' = 4.773 + 4.12x - 8.892x^2, \quad F_3'' = 4.12 - 17.784x, \quad F_3''' = -17.784.$$

Ahora $c_2 = F_3''(0)/2 = 4.12/2 = 2.06$. A partir de aquí, salvo que aparezca el subíndice RK, todas las comparaciones usan F_3.

3. Por qué no se divide dos veces entre factoriales.

Los parámetros publicados de Wagner ε, ρ y θ son ya los coeficientes c_1, c_2, c_3. Si $\rho = 2.06$, el término es $2.06x^2$. Dividir de nuevo entre 2 daría $1.03x^2$, que no corresponde al desarrollo declarado.

4. Qué es el resto.

Para una función con suficientes derivadas, el resto de Lagrange es

$$R_N(x) = \frac{F^{(N+1)}(\xi)}{(N+1)!}x^{N+1}, \qquad 0 < \xi < x.$$

En nuestro cúbico, para $N = 3$, $F_3^{(4)}(\xi) = 0$ y $R_3 = 0x^4/24 = 0$. Para $N = 2$, $F_3''' = -17.784$, así que $R_2 = (-17.784/6)x^3 = -2.964x^3$. Más adelante restaremos polinomios para obtener también R_0 y R_1 exactamente.

Qué acabamos de hacer. El desarrollo finito es exacto respecto del modelo elegido. Esto no demuestra que un líquido real tenga derivadas de orden cuatro nulas ni que el modelo carezca de error físico.

13 d) Construir cada aproximación y hacer una cuenta completa

1. Truncar significa dejar de sumar después de una potencia.

$$
\begin{aligned}
P_0(x) &= -3.8685,\\
P_1(x) &= -3.8685 + 4.773x,\\
P_2(x) &= -3.8685 + 4.773x + 2.06x^2,\\
P_3(x) &= -3.8685 + 4.773x + 2.06x^2 - 2.964x^3 = F_3(x).
\end{aligned}
$$

Orden 0 tiene 1 término; orden 1, 2 términos; orden 2, 3 términos; orden 3, 4 términos. Contamos la constante. Los coeficientes no se reajustan.

2. Elegir una composición para practicar: $x = 0.1$.

Primero las potencias: $x^0 = 1$, $x^1 = 0.1$, $x^2 = (0.1)(0.1) = 0.01$, $x^3 = (0.01)(0.1) = 0.001$.

3. Multiplicar cada coeficiente por su potencia.

$$
\begin{aligned}
t_0 &= c_0x^0 = (-3.8685)(1) = -3.8685,\\
t_1 &= c_1x = (4.773)(0.1) = 0.4773,\\
t_2 &= c_2x^2 = (2.06)(0.01) = 0.0206,\\
t_3 &= c_3x^3 = (-2.964)(0.001) = -0.002964.
\end{aligned}
$$

4. Sumar de forma acumulativa.

$$
\begin{aligned}
P_0 &= t_0 = -3.8685,\\
P_1 &= P_0 + t_1 = -3.8685 + 0.4773 = -3.3912,\\
P_2 &= P_1 + t_2 = -3.3912 + 0.0206 = -3.3706,\\
P_3 &= P_2 + t_3 = -3.3706 - 0.002964 = -3.373564.
\end{aligned}
$$

5. Regresar del logaritmo al coeficiente.

$\gamma_N = e^{P_N}$. Por ejemplo $\gamma_3 = e^{-3.373564} \simeq 0.034267291$; esta operación se evalúa con la función exponencial de una calculadora o del programa. La actividad es $a_{\mathrm{Al},3} = x\gamma_3 \simeq 0.1(0.034267291) = 0.0034267291$.

Atención: P_3, γ_3 y $a_{\mathrm{Al},3}$ son tres resultados diferentes. No se grafica uno con el nombre de otro. La extensión posterior explica cómo calcular la exponencial mediante una serie propia.

14 d) Del ejemplo a todos los puntos de cálculo

1. Construir la malla sin recurrir a una tabla experimental.

Elegimos $M = 10001$ puntos entre 0 y 0.3872. Entre 10001 puntos hay 10000 separaciones, por eso

$$h = \frac{0.3872 - 0}{10001 - 1} = \frac{0.3872}{10000} = 0.00003872.$$

Numeramos desde $j = 0$ hasta $j = 10000$: $x_j = 0 + jh$. Los primeros son $x_0 = 0$, $x_1 = 0.00003872$, $x_2 = 0.00007744$. El último es $x_{10000} = 10000(0.00003872) = 0.3872$. Python usa `linspace(0, 0.3872, 10001)` para generar esa malla.

2. Repetir exactamente las operaciones del ejemplo.

Para cada x_j: calcular x_j^2, x_j^3, los cuatro productos $c_k x_j^k$ y las cuatro sumas acumuladas $P_N(x_j)$. Después calcular las exponenciales y las actividades. No se buscan nuevos coeficientes para cada punto.

3. Ver todos los productos de las composiciones ilustrativas.

La tabla siguiente se genera desde el programa, sin teclear manualmente sus resultados:

x	c_0	$c_1 x$	$c_2 x^2$	$c_3 x^3$
0.0100	-3.868500000	0.047730000	0.000206000	-0.000002964
0.1000	-3.868500000	0.477300000	0.020600000	-0.002964000
0.2000	-3.868500000	0.954600000	0.082400000	-0.023712000
0.3000	-3.868500000	1.431900000	0.185400000	-0.080028000
0.3872	-3.868500000	1.848105600	0.308843110	-0.172061714

Para obtener P_2 de una fila se suman sus tres primeros productos; para P_3 se suman los cuatro.

x	P_1	P_2	$F_3 = P_3$
0.010000	-3.820770	-3.820564	-3.820567
0.100000	-3.391200	-3.370600	-3.373564
0.200000	-2.913900	-2.831500	-2.855212
0.300000	-2.436600	-2.251200	-2.331228
0.387200	-2.020394	-1.711551	-1.883613

4. Localizar todas las cuentas repetidas.

`output/datos/desglose_malla.csv` contiene una fila por composición y orden: 10001 composiciones por 4 órdenes, es decir, 40004 filas. Guarda las potencias, productos, sumas, diferencia, valor absoluto, cuadrado del error y error relativo. Así cada resultado agregado se puede reconstruir sin transcribir decenas de miles de filas al PDF.

Los decimales impresos se redondean para lectura. El programa conserva la precisión de punto flotante y no reutiliza los valores ya redondeados del documento.

15 d) Error absoluto: restar primero, tomar valor absoluto después

1. Fijar la referencia y el signo de la resta.

La referencia es F_3. Definimos $\Delta_N = P_N - F_3$; puede ser positiva o negativa. El error absoluto es $E_N = |\Delta_N|$. El valor absoluto elimina el signo: $|-2| = 2$ y $|2| = 2$.

2. Restar para orden 0.

$$\begin{aligned}\Delta_0 &= -3.8685 - [-3.8685 + 4.773x + 2.06x^2 - 2.964x^3] \\ &= -3.8685 + 3.8685 - 4.773x - 2.06x^2 + 2.964x^3 \\ &= -4.773x - 2.06x^2 + 2.964x^3.\end{aligned}$$

En nuestro intervalo, $4.773 + x(2.06 - 2.964x) > 0$, pues $2.06 - 2.964(0.3872) > 0$. Por tanto $\Delta_0 \leq 0$ y $E_0 = 4.773x + 2.06x^2 - 2.964x^3$.

3. Restar para orden 1.

Los términos constante y lineal se cancelan:

$$\Delta_1 = (-3.8685 + 4.773x) - F_3 = -2.06x^2 + 2.964x^3 = -x^2(2.06 - 2.964x).$$

Como el paréntesis es positivo en el intervalo, $E_1 = x^2(2.06 - 2.964x)$.

4. Restar para órdenes 2 y 3.

$$\Delta_2 = (-3.8685 + 4.773x + 2.06x^2) - F_3 = 2.964x^3 \geq 0,$$

por lo que $E_2 = 2.964x^3$. Para orden 3, $\Delta_3 = F_3 - F_3 = 0$, así que $E_3 = 0$.

5. Hacer las restas en $x = 0.1$.

$$\begin{aligned}\Delta_0 &= -3.8685 - (-3.373564) = -0.494936, & E_0 &= 0.494936, \\ \Delta_1 &= -3.3912 - (-3.373564) = -0.017636, & E_1 &= 0.017636, \\ \Delta_2 &= -3.3706 - (-3.373564) = 0.002964, & E_2 &= 0.002964, \\ \Delta_3 &= -3.373564 - (-3.373564) = 0, & E_3 &= 0.\end{aligned}$$

Un signo negativo indica subestimación del logaritmo; uno positivo, sobreestimación. Como la exponencial es creciente, conserva el sentido de esa comparación.

16 d) Error del coeficiente: por qué aparece otra exponencial

1. Escribir la definición de error relativo porcentual.

El coeficiente aproximado es $\gamma_N = e^{P_N}$ y el de referencia es $\gamma_{\text{ref}} = e^{F_3} > 0$. Definimos

$$e_{\gamma,N} = 100\frac{|\gamma_N - \gamma_{\text{ref}}|}{\gamma_{\text{ref}}}.$$

Dividir entre la referencia expresa el tamaño del error en relación con ella. Multiplicar por 100 lo convierte a porcentaje.

2. Sustituir y simplificar sin saltos.

$$\begin{aligned} e_{\gamma,N} &= 100\frac{|e^{P_N} - e^{F_3}|}{e^{F_3}} = 100\left|\frac{e^{P_N}}{e^{F_3}} - \frac{e^{F_3}}{e^{F_3}}\right| \\ &= 100|e^{P_N - F_3} - 1| = 100|e^{\Delta_N} - 1|. \end{aligned}$$

Usamos $e^a/e^b = e^{a-b}$ y $e^{F_3}/e^{F_3} = 1$. No es correcto llamar a $|P_N - F_3|$ "porcentaje de error de γ".

3. Cuenta numérica completa en $x = 0.1$.

Orden 0:

$$e_{\gamma,0} = 100|e^{-0.494936} - 1| \simeq 100|0.6096099211 - 1| \simeq 39.03900789\,\%.$$

Orden 1:

$$e_{\gamma,1} = 100|e^{-0.017636} - 1| \simeq 100|0.9825186040 - 1| \simeq 1.74813960\,\%.$$

Orden 2:

$$e_{\gamma,2} = 100|e^{0.002964} - 1| \simeq 100|1.0029683970 - 1| \simeq 0.29683970\,\%.$$

Orden 3:

$$e_{\gamma,3} = 100|e^{-0.000000} - 1| \simeq 100|1.0000000000 - 1| \simeq 0.00000000\,\%.$$

4. Diferencia entre dos medidas.

El error absoluto de P_1 es 0.017636, mientras que su error relativo en γ es el porcentaje anterior. Son magnitudes diferentes. Para Δ muy pequeño, $e^{\Delta} - 1 \simeq \Delta$, pero eso es una aproximación local, no una igualdad universal.

5. Evaluación estable.

El programa usa `expm1(delta)`, que calcula $e^{\Delta} - 1$ con buena precisión cuando Δ es cercano a cero. También obtiene Δ_N sumando los términos omitidos con signo opuesto. Restar dos números casi iguales, $P_N - F_3$, puede perder cifras por cancelación; sumar directamente lo omitido evita esa resta.

17 d) Métricas globales: máximo y RMSE, operación por operación

1. Error máximo.

Calculamos $E_{N,j} = |P_N(x_j) - F_3(x_j)|$ para cada punto y elegimos el mayor. Eso significa $E_{\max,N} = \max_{0\le j\le 10000} E_{N,j}$. No sumamos los errores para esta métrica.

2. RMSE: elevar, sumar, dividir y sacar raíz.

RMSE significa raíz del promedio de los errores al cuadrado:

$$S_N = \sum_{j=0}^{10000} \Delta_{N,j}^2, \qquad \mathrm{MSE}_N = \frac{S_N}{10001}, \qquad \mathrm{RMSE}_N = \sqrt{\mathrm{MSE}_N}.$$

Elevar al cuadrado hace positivas las contribuciones; así errores de signo opuesto no se cancelan. Dividir entre 10001 da el promedio. La raíz devuelve la escala original del error.

3. Ejemplo pequeño para comprender la operación.

Supongamos, solo para practicar, errores 0, 0.01, −0.02. Sus cuadrados son 0, 0.0001, 0.0004. La suma es 0.0005; el promedio es $0.0005/3 \simeq 0.0001666667$; la raíz es 0.01290994. El máximo absoluto es 0.02. Este ejemplo de tres datos no sustituye la malla real.

4. Sumas intermedias de la malla real.

Orden	S_N	$S_N/10001$	$\sqrt{S_N/10001}$
0	13024.7992623275	1.3023496913	1.1412053677
1	55.9370959053	0.0055931503	0.0747873671
2	42.3079947790	0.0042303764	0.0650413441
3	0.0000000000	0.0000000000	0.0000000000

Cada suma usa los 10001 cuadrados guardados en el CSV. El programa calcula todos los términos; la tabla muestra el resultado de agregarlos.

5. Resultados finales.

Orden	Términos	$E_{\max}$	RMSE	$e_{\gamma,\max}$ (%)
0	1	1.984887	1.141205	86.2604
1	2	0.136781	0.074787	12.7839
2	3	0.172062	0.065041	18.7751
3	4	0.000000	0.000000	0.0000

Para cuantificar el cambio de RMSE de orden 1 a 2 se calcula

$$100\frac{0.074787 - 0.065041}{0.074787} \simeq 13.0\,\%.$$

Para el aumento del error máximo: $100(0.172062 - 0.136781)/0.136781 \simeq 25.8\,\%$. Los números mostrados están redondeados; las métricas del programa usan todos sus decimales.

18 d) Por qué agregar un término puede empeorar el máximo

1. Cerca de cero.

$E_1 = 2.06x^2 - 2.964x^3 = x^2(2.06 - 2.964x)$. Al acercarse x a cero, el paréntesis se acerca a 2.06, de modo que E_1 decrece como x^2. En cambio $E_2 = 2.964x^3$ decrece como x^3. Esto se escribe $E_1 = O(x^2)$ y $E_2 = O(x^3)$: describe el orden de disminución local.

2. Buscar dónde se igualan los errores.

Para $x > 0$, planteamos $E_1 = E_2$:

$$\begin{aligned} 2.06x^2 - 2.964x^3 &= 2.964x^3, \\ 2.06x^2 &= 2.964x^3 + 2.964x^3 = 5.928x^3, \\ 2.06 &= 5.928x, \qquad \text{dividiendo entre } x^2 > 0, \\ x &= \frac{2.06}{5.928} \simeq 0.347503374. \end{aligned}$$

$x = 0$ también iguala los errores, pero se separó antes de dividir. Para $x > 0.347503374$ dentro del intervalo, $E_2 > E_1$.

3. Explicación de los signos.

El término $+2.06x^2$ sube el valor y el término $-2.964x^3$ lo baja. Al omitir ambos, parte de sus efectos opuestos se cancela. Al retener solo el positivo, se pierde esa compensación. Por eso mayor grado no garantiza menor error en todos los puntos de un intervalo finito.

4. Dónde están los máximos de E_1 y E_2.

Derivamos: $E_1' = 4.12x - 8.892x^2 = x(4.12 - 8.892x)$. En $0 < x \leq 0.3872$, el paréntesis es positivo porque $4.12 - 8.892(0.3872) > 0$. Entonces E_1 aumenta y su máximo está en el extremo. $E_2' = 8.892x^2 > 0$ para $x > 0$, así que también aumenta. $E_0' = F_3' > 0$ en este intervalo, como se comprueba más adelante. Para $E_3 = 0$, todos los puntos comparten el máximo cero.

5. Una comparación distinta de la del artículo.

Aquí P_2 es el corte cuadrático del mismo cúbico. El formalismo cuadrático independiente de los autores tiene su propia construcción. No deben identificarse ambas comparaciones.

19 d) Tolerancia de 1 por ciento: resolver la desigualdad

1. Qué buscamos.

Queremos el intervalo que empieza en cero y satisface $e_{\gamma,N} \leq 1$. Usamos el error del coeficiente, no el del logaritmo.

2. Orden 2: solución algebraica.

Como $\Delta_2 = 2.964x^3 \geq 0$, $|e^{\Delta_2} - 1| = e^{\Delta_2} - 1$:

$$\begin{aligned} 100(e^{2.964x^3} - 1) &\leq 1, \\ e^{2.964x^3} - 1 &\leq 0.01, \\ e^{2.964x^3} &\leq 1.01, \\ 2.964x^3 &\leq \ln(1.01), \\ x^3 &\leq \frac{\ln(1.01)}{2.964}, \\ x &\leq \left[\frac{\ln(1.01)}{2.964}\right]^{1/3} \simeq 0.14973. \end{aligned}$$

Aplicar ln y raíz cúbica conserva el sentido porque ambas funciones son crecientes en los valores utilizados.

3. Orden 1: ecuación de la frontera.

$\Delta_1 = -E_1 \leq 0$; por ello $|e^{-E_1} - 1| = 1 - e^{-E_1}$. En la frontera:

$$100(1 - e^{-E_1}) = 1 \Rightarrow e^{-E_1} = 0.99 \Rightarrow -E_1 = \ln(0.99) \Rightarrow E_1 = -\ln(0.99).$$

Sustituimos E_1: $2.06x^2 - 2.964x^3 = -\ln(0.99)$. Resolvemos numéricamente; como E_1 aumenta en el intervalo, hay una sola frontera.

4. Bisección: instrucciones exactas.

Empezamos con extremo inferior $l = 0$ y superior $u = 0.3872$. El punto medio es $m = (l + u)/2$. Si su error es menor o igual a 1 %, reemplazamos l por m; si supera 1 %, reemplazamos u por m. Repetimos 70 veces. Al dividir a la mitad el intervalo se encierra cada vez más la frontera. Cerca de la precisión de máquina, las últimas iteraciones pueden repetir el mismo número; no se obtienen 70 cifras correctas.

La frontera lineal es $x \simeq 0.07389$. La tabla siguiente muestra los primeros pasos; el CSV guarda las 70 iteraciones de cada orden.

Paso	l	u	m	Error (%)	Decisión
1	0.00000000	0.38720000	0.19360000	5.41801	bajar superior
2	0.00000000	0.19360000	0.09680000	1.64770	bajar superior
3	0.00000000	0.09680000	0.04840000	0.44796	subir inferior
4	0.04840000	0.09680000	0.07260000	0.96764	subir inferior
5	0.07260000	0.09680000	0.08470000	1.28937	bajar superior
6	0.07260000	0.08470000	0.07865000	1.12372	bajar superior

20 e) Leer las gráficas: qué representa cada eje

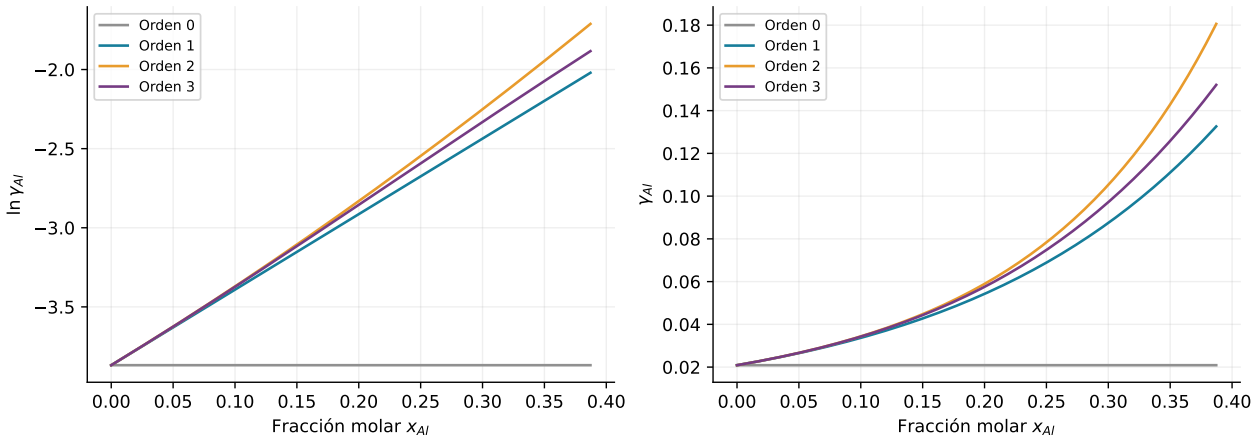


Figura 1. En ambas gráficas el eje horizontal es x, fracción molar de Al. A la izquierda, cada altura es $P_N(x)$, un logaritmo. A la derecha, cada altura es $e^{P_N(x)}$, el coeficiente. Las curvas se generan evaluando las fórmulas en la malla. No son curvas ajustadas aquí a mediciones. Orden 3 es la referencia.

Para reconstruir un punto, elegir por ejemplo $x = 0.1$: a la izquierda el punto cúbico tiene altura -3.373564; a la derecha, $e^{-3.373564}$. Unir puntos consecutivos permite visualizar el comportamiento, pero las operaciones que los producen son las mismas que en la cuenta manual.

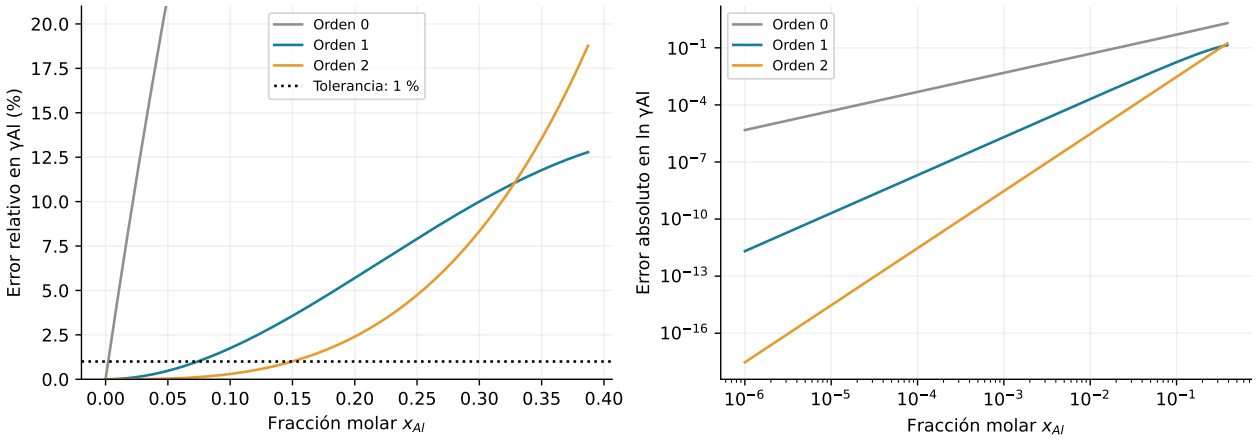


Figura 2. A la izquierda se representa $100|e^{\Delta_N} - 1|$. La línea punteada indica 1 %. El eje vertical se limita a 21 % para distinguir órdenes 1 y 2; el orden 0 sale del recuadro. A la derecha se representa $|\Delta_N|$ con ambos ejes logarítmicos. Una malla geométrica separada de 1000 puntos positivos entre 10^{-6} y 0.3872 permite ver la región diluida. No se incluye cero porque $\ln 0$ no existe. Orden 3 tampoco se dibuja allí: su error es cero.

Una escala logarítmica coloca a distancias iguales cambios por factores, como 10^{-3}, 10^{-2} y 10^{-1}. Si $E \simeq Cx^p$, entonces $\ln E \simeq \ln C + p \ln x$; la pendiente en esta gráfica es p. Por eso se reconocen órdenes locales 1, 2 y 3 en los errores de P_0, P_1, P_2, respectivamente.

21 c) Por qué el hierro necesita su propia derivación

1. La restricción termodinámica.

A temperatura y presión constantes, Gibbs-Duhem vincula los potenciales químicos. Para el binario y los coeficientes de actividad:

$$x_{\mathrm{Fe}}\, d\ln\gamma_{\mathrm{Fe}} + x_{\mathrm{Al}}\, d\ln\gamma_{\mathrm{Al}} = 0.$$

Esta es una identidad termodinámica que usamos como fundamento, al igual que la relación entre potencial químico y actividad; no es una ecuación ajustada con los datos de la malla. Su significado es que los cambios de ambas especies no pueden elegirse de forma independiente.

2. Cambiar a la notación del trabajo.

$x_{\mathrm{Fe}} = 1 - x$, $x_{\mathrm{Al}} = x$, $H = \ln\gamma_{\mathrm{Fe}}$, $F = \ln\gamma_{\mathrm{Al}}$. Sustituyendo:

$$(1-x)\, dH + x\, dF = 0.$$

Como H y F dependen de x, dividimos formalmente entre dx para escribir las derivadas:

$$(1-x)H'(x) + xF'(x) = 0.$$

3. Despejar la derivada del hierro.

Restamos xF' a ambos lados y dividimos entre $1-x$ (no nulo en nuestro intervalo):

$$(1-x)H' = -xF' \quad \Rightarrow \quad H' = -\frac{x}{1-x}F'.$$

Para una aproximación de Al P_N definimos el hierro correspondiente H_N con $H'_N = -xP'_N/(1-x)$.

4. Fijar la condición de frontera.

En $x = 0$ hay hierro puro. En la escala de referencia adoptada, $\gamma_{\mathrm{Fe}}(0) = 1$; entonces $H_N(0) = \ln 1 = 0$. Integrar la derivada desde cero hasta x da

$$H_N(x) - H_N(0) = \int_0^x H'_N(t)\, dt = -\int_0^x \frac{t}{1-t} P'_N(t)\, dt.$$

El símbolo t es una variable auxiliar de integración; se usa para no confundirla con el límite superior x. Como $H_N(0) = 0$:

$$\boxed{H_N(x) = -\int_0^x \frac{t}{1-t} P'_N(t)\, dt.}$$

Qué acabamos de hacer. Para cada truncamiento del aluminio calculamos un hierro compatible. Usar siempre el hierro cúbico con un aluminio lineal rompería esta construcción.

22 c) Integrar Gibbs-Duhem: resolver las tres integrales

1. Derivar el polinomio del aluminio.

$P_3(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3$ da $P_3'(t) = c_1 + 2c_2 t + 3c_3 t^2$. Multiplicamos por $-t/(1-t)$ y separamos la integral:

$$H_3 = -c_1 \int_0^x \frac{t}{1-t} dt - 2c_2 \int_0^x \frac{t^2}{1-t} dt - 3c_3 \int_0^x \frac{t^3}{1-t} dt.$$

2. Primera fracción.

Como $t = 1-(1-t)$, $t/(1-t) = 1/(1-t) - 1$. La primitiva de $1/(1-t)$ es $-\ln(1-t)$: al derivarla, el -1 exterior se multiplica por la derivada -1 de $1-t$, dando signo positivo. Por tanto

$$\int_0^x \frac{t}{1-t} dt = [-\ln(1-t) - t]_0^x = -\ln(1-x) - x,$$

ya que en $t = 0$ se obtiene $-\ln 1 - 0 = 0$.

3. Segunda fracción.

Verificamos $(1-t)(1+t) = 1 - t^2$, luego $1 - (1-t)(1+t) = t^2$. Al dividir entre $1-t$:

$$\frac{t^2}{1-t} = \frac{1}{1-t} - 1 - t.$$

Integramos término a término; la primitiva de t es $t^2/2$:

$$\int_0^x \frac{t^2}{1-t} dt = [-\ln(1-t) - t - t^2/2]_0^x = -\ln(1-x) - x - x^2/2.$$

4. Tercera fracción.

$(1-t)(1+t+t^2) = 1 - t^3$. Entonces $t^3/(1-t) = 1/(1-t) - 1 - t - t^2$. La primitiva de t^2 es $t^3/3$:

$$\int_0^x \frac{t^3}{1-t} dt = [-\ln(1-t) - t - t^2/2 - t^3/3]_0^x = -\ln(1-x) - x - x^2/2 - x^3/3.$$

5. Multiplicar por los coeficientes exteriores.

Cada integral tiene un signo negativo que se cancela con el signo exterior:

$$\begin{aligned} H_3 = c_1[\ln(1-x) + x] \; + \; 2c_2[\ln(1-x) + x + x^2/2] \\ + \, 3c_3[\ln(1-x) + x + x^2/2 + x^3/3]. \end{aligned}$$

El valor de c_3 sigue siendo negativo; cancelar los signos de integración no lo convierte en positivo.

23 c) Hierro de cada orden y cancelación del logaritmo

1. Escribir los cuatro casos.

Con $c_1 = 4.773$, $2c_2 = 2(2.06) = 4.12$ y $3c_3 = 3(-2.964) = -8.892$:

$$\begin{aligned} H_0 &= 0, \\ H_1 &= 4.773[\ln(1-x) + x], \\ H_2 &= H_1 + 4.12[\ln(1-x) + x + x^2/2], \\ H_3 &= H_2 - 8.892[\ln(1-x) + x + x^2/2 + x^3/3]. \end{aligned}$$

$H_0 = 0$ porque la derivada de P_0 es cero. En todos los casos, sustituir $x = 0$ da cero.

2. Agrupar en H_3 cada tipo de término.

$$\begin{aligned} \ln(1-x): &\quad 4.773 + 4.12 - 8.892 = 0.001, \\ x: &\quad 4.773 + 4.12 - 8.892 = 0.001, \\ x^2: &\quad 4.12/2 - 8.892/2 = 2.06 - 4.446 = -2.386, \\ x^3: &\quad -8.892/3 = -2.964. \end{aligned}$$

Así $H_3 = 0.001[\ln(1-x) + x] - 2.386x^2 - 2.964x^3$.

3. Repetir con $c_2 = 2.0595$ sin redondear.

Ahora $2c_2 = 4.119$. El coeficiente del logaritmo y del término lineal es $4.773 + 4.119 - 8.892 = 0$. El de x^2 es $4.119/2 - 8.892/2 = 2.0595 - 4.446 = -2.3865$. Por tanto $H_{\mathrm{RK}} = -2.3865x^2 - 2.964x^3$.

4. Comprobarlo directamente desde la energía.

Usando $H_{\mathrm{RK}} = f - xf'$:

$$\begin{aligned} f - xf' &= [-3.8685x + 2.3865x^2 + 1.482x^3] \\ &\quad - [-3.8685x + 4.773x^2 + 4.446x^3] \\ &= (-3.8685 + 3.8685)x + (2.3865 - 4.773)x^2 \\ &\quad + (1.482 - 4.446)x^3 = -2.3865x^2 - 2.964x^3. \end{aligned}$$

La ruta parcial molar y la ruta Gibbs-Duhem coinciden para los mismos coeficientes. El pequeño término logarítmico de H_3 se debe al redondeo, no a una nueva interacción física identificada.

24 c) Fórmula general y precisión cerca de cero

1. Reconocer el patrón de las tres integrales.

Para un entero $k \geq 1$:

$$\frac{t^k}{1-t} = \frac{1}{1-t} - \sum_{j=0}^{k-1} t^j.$$

Se verifica multiplicando por $1-t$: los términos intermedios de $(1-t)(1+t+\cdots+t^{k-1})$ se cancelan y queda $1-t^k$. La derivada de $c_k t^k$ es $kc_k t^{k-1}$; al multiplicar por $t/(1-t)$ queda $kc_k t^k/(1-t)$. Integrar y aplicar la frontera da

$$H_N = \sum_{k=1}^{N} kc_k \left[\ln(1-x) + \sum_{j=1}^{k} \frac{x^j}{j} \right].$$

Para $k = 1, 2, 3$ recuperamos exactamente los corchetes ya calculados.

2. De dónde sale la serie del logaritmo.

Para $|t| < 1$, la suma geométrica es $1/(1-t) = 1+t+t^2+\cdots$. Integramos entre 0 y x (con $|x| < 1$):

$$-\ln(1-x) = x + \frac{x^2}{2} + \frac{x^3}{3} + \cdots.$$

Por tanto $\ln(1-x) = -\sum_{j=1}^{\infty} x^j/j$. Si sumamos los primeros k términos positivos, se cancelan los primeros k negativos:

$$\ln(1-x) + \sum_{j=1}^{k} \frac{x^j}{j} = -\sum_{j=k+1}^{\infty} \frac{x^j}{j}.$$

3. Por qué el programa cambia de forma para $x < 0.1$.

Para x muy pequeño, $\ln(1-x) \simeq -x$. Sumarle x resta números casi iguales y puede borrar cifras significativas en la computadora. La cola de la derecha suma números del mismo signo y evita esa cancelación.

El programa conserva la cola hasta $j = 32$. Para $0 \leq x < 0.1$, el error omitido es menor o igual a

$$\sum_{j=33}^{\infty} \frac{x^j}{j} \leq \frac{1}{33} \sum_{j=33}^{\infty} x^j = \frac{x^{33}}{33(1-x)} < \frac{10^{-33}}{33(0.9)} < 10^{-32}.$$

Fuera de esa región se evalúa la expresión con `log1p(-x)`, que calcula $\ln(1-x)$. Esta cota se refiere a truncar la cola, no a eliminar todo error de redondeo de la computadora.

25 f) Energía de exceso: reconstruirla desde Al y Fe

1. Ponderar las contribuciones.

La energía molar adimensional de exceso del binario se expresa como $x \ln \gamma_{\mathrm{Al}} + (1-x) \ln \gamma_{\mathrm{Fe}}$. Para orden N:

$$g_N^E = RT[xP_N + (1-x)H_N].$$

No se suman directamente $P_N + H_N$: cada especie aporta en proporción a su fracción molar.

2. Verificar la fórmula con las propiedades parciales originales.

Insertamos $F = f + (1-x)f'$ y $H = f - xf'$:

$$\begin{aligned} xF + (1-x)H &= x[f + (1-x)f'] + (1-x)[f - xf'] \\ &= xf + x(1-x)f' + (1-x)f - x(1-x)f' \\ &= [x + (1-x)]f + 0 \cdot f' = 1 \cdot f = f. \end{aligned}$$

Multiplicando por RT recuperamos $RTf = g^E$. Para los coeficientes redondeados se usa la pareja F_3, H_3, que satisface Gibbs-Duhem, aunque difiere ligeramente del RK original.

3. Ejemplo numérico detallado en $x = 0.1$.

Primero, $1 - x = 1 - 0.1 = 0.9$ y $\ln(1-x) = \ln(0.9) \simeq -0.105360515658$. Los tres corchetes, calculados uno después de otro, son

$$\begin{aligned} s_1 &= \ln(0.9) + 0.1 \simeq -0.005360515658, \\ s_2 &= s_1 + 0.1^2/2 = s_1 + 0.005 \simeq -0.000360515658, \\ s_3 &= s_2 + 0.1^3/3 = s_2 + 0.000333333333 \simeq -0.000027182324. \end{aligned}$$

Multiplicamos por los factores exteriores:

$$4.773 s_1 \simeq -0.025585741235, \quad 4.12 s_2 \simeq -0.001485324510, \quad -8.892 s_3 \simeq 0.000241705229.$$

Sumamos esas tres contribuciones: $H_3 \simeq -0.026829360516$. Ahora ponderamos:

$$xP_3 = (0.1)(-3.373564) = -0.3373564000.$$

$$(1-x)H_3 = (0.9)(-0.026829360516) \simeq -0.024146424464.$$

La suma es $xP_3 + (1-x)H_3 \simeq -0.361502824464$. Finalmente,

$$g_3^E = (15572.986483514)(-0.361502824464) \simeq -5629.67932214 \text{ J/mol}.$$

Dividimos entre 1000: $g_3^E \simeq -5.62967932$ kJ/mol.

4. Repetir con otros órdenes.

Para $N = 1$, se usan P_1 y H_1; para $N = 2$, P_2 y H_2. Las mismas multiplicaciones por x, $1 - x$ y RT producen g_1^E y g_2^E. El programa registra estas propiedades para cada punto en `malla.csv`; el máximo de $|g_N^E - g_3^E|$ se obtiene igual que el máximo de cualquier otra lista de errores.

26 f) Cuenta completa en el extremo de mayor composición

1. Potencias y productos en $x = 0.3872$.

$$x^2 = (0.3872)(0.3872) = 0.14992384, \quad x^3 = (0.14992384)(0.3872) = 0.058050510848.$$

$$\begin{aligned} t_0 &= -3.8685, \\ t_1 &= 4.773(0.3872) = 1.8481056, \\ t_2 &= 2.06(0.14992384) = 0.3088431104, \\ t_3 &= -2.964(0.058050510848) = -0.172061714153472. \end{aligned}$$

2. Sumas acumuladas.

$$\begin{aligned} P_0 &= -3.8685000, \\ P_1 &= -3.8685000 + 1.8481056 = -2.0203944, \\ P_2 &= -2.0203944 + 0.3088431104 = -1.7115512896, \\ P_3 &= -1.7115512896 + (-0.172061714153472) \simeq -1.883613003753. \end{aligned}$$

3. Exponencial y actividad.

$$\gamma_3 = e^{-1.883613003753} \simeq 0.1520397919, \quad a_{\mathrm{Al},3} = (0.3872)(0.1520397919) \simeq 0.0588698074.$$

4. Hierro y energía.

$1 - x = 0.6128$ y $\ln(0.6128) \simeq -0.489716660556$. Usando la forma agrupada,

$$\begin{aligned} H_3 &= 0.001[\ln(0.6128) + 0.3872] - 2.386(0.14992384) - 2.964(0.058050510848) \\ &\simeq -0.000102516661 - 0.357718282240 - 0.172061714153 \\ &\simeq -0.529882513054. \end{aligned}$$

Entonces $xP_3 \simeq -0.729334955053$ y $(1-x)H_3 \simeq -0.324712004000$. Su suma es -1.054046959053. Multiplicando por RT:

$$g_3^E \simeq -16414.66115441 \text{ J/mol} = -16.41466115 \text{ kJ/mol}.$$

Qué indica esta cuenta.

La diferencia entre las aproximaciones es visible tanto en los logaritmos como en sus exponenciales. El coeficiente del Al permanece por debajo de 1. Su actividad también queda por debajo de la fracción molar. El hierro tiene su propia contribución no ideal y por eso debe incluirse para calcular la energía de la mezcla.

27 Extensión: distinguir dos aproximaciones que no son iguales

1. Método utilizado hasta ahora.

Primero aproximamos el logaritmo con P_N; después usamos $\gamma_N = e^{P_N}$. Si $N = 3$, $P_3 = F_3$ y $e^{P_3} = e^{F_3}$ coincide con la referencia. Por eso su error es cero.

2. Una pregunta matemática nueva.

Ahora queremos aproximar directamente $\gamma_{\text{ref}}(x) = e^{F_3(x)}$ mediante un polinomio:

$$Q_m(x) = b_0 + b_1 x + b_2 x^2 + \cdots + b_m x^m.$$

Los b_n no son los c_n del logaritmo. Sus valores deben calcularse. Q_3 no es e^{P_3} y no tiene por qué tener error cero.

3. Por qué aparecen infinitos términos.

La exponencial tiene el desarrollo $e^u = 1 + u + u^2/2! + u^3/3! + \cdots$. Escribimos $F_3 = c_0 + v(x)$, donde $v = c_1 x + c_2 x^2 + c_3 x^3$. Como $e^{c_0+v} = e^{c_0} e^v$:

$$e^{F_3} = e^{c_0} \left[1 + v + \frac{v^2}{2!} + \frac{v^3}{3!} + \cdots \right].$$

Aunque v es cúbico, sus potencias producen grados cada vez mayores: v^2 puede llegar a grado 6, v^3 a grado 9, etc. Por ejemplo, multiplicar $c_3 x^3$ por sí mismo r veces produce $c_3^r x^{3r}$. La operación exponencial no corta en grado tres.

4. La expansión sigue usando los mismos datos iniciales.

Los únicos parámetros son $c_0 = -3.8685$, $c_1 = 4.773$, $c_2 = 2.06$ y $c_3 = -2.964$. No hay una nueva tabla de coeficientes experimentales para la serie infinita. Los b_n se deducen matemáticamente de estos cuatro números.

5. Comprobar la primera derivada directamente.

$\gamma = e^{F_3}$ implica, por regla de la cadena, $\gamma' = F_3' e^{F_3} = F_3' \gamma$. En cero:

$$b_0 = \gamma(0) = e^{c_0}, \qquad b_1 = \gamma'(0) = F_3'(0)\gamma(0) = c_1 e^{c_0}.$$

Para seguir sin calcular muchas derivadas compuestas, derivaremos una recurrencia: una regla que usa coeficientes ya conocidos para obtener el siguiente.

28 Extensión: derivar la recurrencia sin saltos de índices

1. Escribir la serie y su derivada.

$$\gamma = b_0 + b_1x + b_2x^2 + b_3x^3 + b_4x^4 + \cdots ,$$
$$\gamma' = b_1 + 2b_2x + 3b_3x^2 + 4b_4x^3 + \cdots .$$

La derivada del exponente es $F_3' = c_1 + 2c_2x + 3c_3x^2$. Usamos $\gamma' = F_3'\gamma$.

2. Distribuir los tres factores del producto.

$$\begin{aligned} F_3'\gamma = {} & c_1(b_0 + b_1x + b_2x^2 + b_3x^3 + \cdots) \\ & + 2c_2x(b_0 + b_1x + b_2x^2 + \cdots) \\ & + 3c_3x^2(b_0 + b_1x + b_2x^2 + \cdots). \end{aligned}$$

Agrupamos potencias iguales:

$$\begin{aligned} x^0 &: \quad c_1b_0, \\ x^1 &: \quad c_1b_1 + 2c_2b_0, \\ x^2 &: \quad c_1b_2 + 2c_2b_1 + 3c_3b_0, \\ x^3 &: \quad c_1b_3 + 2c_2b_2 + 3c_3b_1. \end{aligned}$$

3. Igualar los coeficientes de ambos lados.

Si dos series representan la misma función alrededor de cero, sus coeficientes de cada potencia son iguales:

$$\begin{aligned} b_1 &= c_1b_0, \\ 2b_2 &= c_1b_1 + 2c_2b_0, \\ 3b_3 &= c_1b_2 + 2c_2b_1 + 3c_3b_0, \\ 4b_4 &= c_1b_3 + 2c_2b_2 + 3c_3b_1. \end{aligned}$$

Dividir la segunda línea entre 2, la tercera entre 3 y la cuarta entre 4 despeja b_2, b_3, b_4.

4. Escribir el patrón para cualquier $n \geq 1$.

En la potencia x^{n-1} del lado izquierdo aparece nb_n. Del derecho pueden contribuir los tres términos de F_3':

$$\boxed{b_0 = e^{c_0}, \qquad b_n = \frac{1}{n} \sum_{k=1}^{\min(3,n)} kc_kb_{n-k}.}$$

El límite $\min(3, n)$ elige el menor de 3 y n: evita pedir b_{-1} o b_{-2}, que no existen. Para $n \geq 3$ se suman tres productos. Para $n = 1$ solo uno y para $n = 2$ dos.

29 Extensión: calcular a mano los primeros coeficientes

1. Coeficiente constante.

$b_0 = e^{c_0} = e^{-3.8685}$. Se evalúa la exponencial; no se toma un número de una tabla del artículo.

2. Coeficiente lineal.

$b_1 = c_1 b_0 = 4.773 b_0$. Hay una multiplicación y la división entre 1 no cambia el resultado.

3. Coeficiente cuadrático.

La ecuación es $2b_2 = c_1 b_1 + 2c_2 b_0$. Sustituyendo $b_1 = c_1 b_0$:

$$2b_2 = c_1(c_1 b_0) + 2c_2 b_0 = (c_1^2 + 2c_2) b_0.$$

Dividimos entre 2: $b_2 = (c_1^2/2 + c_2) b_0$. Numéricamente,

$$c_1^2 = (4.773)(4.773) = 22.781529, \quad c_1^2/2 = 11.3907645,$$

$$c_1^2/2 + c_2 = 11.3907645 + 2.06 = 13.4507645.$$

Así $b_2 = 13.4507645 b_0$. Esta forma es equivalente a la recurrencia; sirve como comprobación.

4. Coeficiente cúbico.

$3b_3 = c_1 b_2 + 2c_2 b_1 + 3c_3 b_0$. Sustituimos las expresiones anteriores:

$$\begin{aligned} 3b_3 &= c_1(c_1^2/2 + c_2) b_0 + 2c_2 c_1 b_0 + 3c_3 b_0 \\ &= (c_1^3/2 + c_1 c_2 + 2c_1 c_2 + 3c_3) b_0 \\ &= (c_1^3/2 + 3c_1 c_2 + 3c_3) b_0. \end{aligned}$$

Dividir entre 3 da $b_3 = (c_1^3/6 + c_1 c_2 + c_3) b_0$. Aquí $4.773^3 = 108.736237917$, $4.773(2.06) = 9.83238$ y $108.736237917/6 = 18.1227063195$. Entonces

$$b_3 = (18.1227063195 + 9.83238 - 2.964) b_0 = 24.9910863195 b_0.$$

5. ¿Y los coeficientes del 4 al 12?

No se necesita volver al artículo. Cada uno se calcula con los tres anteriores. Las páginas siguientes muestran todos los productos, su suma y la división final. Se imprimen decimales redondeados; las cuentas del programa conservan más precisión.

30 Extensión: desglose numérico de todos los coeficientes utilizados

$b_0 = e^{-3.8685} \simeq 0.020889680457$. Este es el punto de partida.

Coeficiente b_1:

$$\begin{aligned} u_1 &= (1)(4.773)b_0 \simeq (1)(4.773)(0.020889680457) \simeq 0.099706444820,\\ s &= (0.099706444820) \simeq 0.099706444820,\\ b_1 &= s/1 \simeq (0.099706444820)/1 \simeq 0.099706444820. \end{aligned}$$

Coeficiente b_2:

$$\begin{aligned} u_1 &= (1)(4.773)b_1 \simeq (1)(4.773)(0.099706444820) \simeq 0.475898861126,\\ u_2 &= (2)(2.06)b_0 \simeq (2)(2.06)(0.020889680457) \simeq 0.086065483482,\\ s &= (0.475898861126) + (0.086065483482) \simeq 0.561964344608,\\ b_2 &= s/2 \simeq (0.561964344608)/2 \simeq 0.280982172304. \end{aligned}$$

Coeficiente b_3:

$$\begin{aligned} u_1 &= (1)(4.773)b_2 \simeq (1)(4.773)(0.280982172304) \simeq 1.341127908407,\\ u_2 &= (2)(2.06)b_1 \simeq (2)(2.06)(0.099706444820) \simeq 0.410790552659,\\ u_3 &= (3)(-2.964)b_0 \simeq (3)(-2.964)(0.020889680457) \simeq -0.185751038621,\\ s &= (1.341127908407) + (0.410790552659) + (-0.185751038621) \simeq 1.566167422445,\\ b_3 &= s/3 \simeq (1.566167422445)/3 \simeq 0.522055807482. \end{aligned}$$

Coeficiente b_4:

$$
\begin{aligned}
u_1 &= (1)(4.773)b_3 \simeq (1)(4.773)(0.522055807482) \simeq 2.491772369109,\\
u_2 &= (2)(2.06)b_2 \simeq (2)(2.06)(0.280982172304) \simeq 1.157646549893,\\
u_3 &= (3)(-2.964)b_1 \simeq (3)(-2.964)(0.099706444820) \simeq -0.886589707340,\\
s &= (2.491772369109) + (1.157646549893) + (-0.886589707340) \simeq 2.762829211662,\\
b_4 &= s/4 \simeq (2.762829211662)/4 \simeq 0.690707302915.
\end{aligned}
$$

Coeficiente b_5:

$$
\begin{aligned}
u_1 &= (1)(4.773)b_4 \simeq (1)(4.773)(0.690707302915) \simeq 3.296745956816,\\
u_2 &= (2)(2.06)b_3 \simeq (2)(2.06)(0.522055807482) \simeq 2.150869926824,\\
u_3 &= (3)(-2.964)b_2 \simeq (3)(-2.964)(0.280982172304) \simeq -2.498493476128,\\
s &= (3.296745956816) + (2.150869926824) + (-2.498493476128) \simeq 2.949122407512,\\
b_5 &= s/5 \simeq (2.949122407512)/5 \simeq 0.589824481502.
\end{aligned}
$$

Coeficiente b_6:

$$
\begin{aligned}
u_1 &= (1)(4.773)b_5 \simeq (1)(4.773)(0.589824481502) \simeq 2.815232250211,\\
u_2 &= (2)(2.06)b_4 \simeq (2)(2.06)(0.690707302915) \simeq 2.845714088012,\\
u_3 &= (3)(-2.964)b_3 \simeq (3)(-2.964)(0.522055807482) \simeq -4.642120240126,\\
s &= (2.815232250211) + (2.845714088012) + (-4.642120240126) \simeq 1.018826098097,\\
b_6 &= s/6 \simeq (1.018826098097)/6 \simeq 0.169804349683.
\end{aligned}
$$

Coeficiente b_7:

$$
\begin{aligned}
u_1 &= (1)(4.773)b_6 \simeq (1)(4.773)(0.169804349683) \simeq 0.810476161036,\\
u_2 &= (2)(2.06)b_5 \simeq (2)(2.06)(0.589824481502) \simeq 2.430076863790,\\
u_3 &= (3)(-2.964)b_4 \simeq (3)(-2.964)(0.690707302915) \simeq -6.141769337524,\\
s &= (0.810476161036) + (2.430076863790) + (-6.141769337524) \simeq -2.901216312699,\\
b_7 &= s/7 \simeq (-2.901216312699)/7 \simeq -0.414459473243.
\end{aligned}
$$

Coeficiente b_8:

$$
\begin{aligned}
u_1 &= (1)(4.773)b_7 \simeq (1)(4.773)(-0.414459473243) \simeq -1.978215065787,\\
u_2 &= (2)(2.06)b_6 \simeq (2)(2.06)(0.169804349683) \simeq 0.699593920693,\\
u_3 &= (3)(-2.964)b_5 \simeq (3)(-2.964)(0.589824481502) \simeq -5.244719289519,\\
s &= (-1.978215065787) + (0.699593920693) + (-5.244719289519) \simeq -6.523340434613,\\
b_8 &= s/8 \simeq (-6.523340434613)/8 \simeq -0.815417554327.
\end{aligned}
$$

Coeficiente b_9:

$$
\begin{aligned}
u_1 &= (1)(4.773)b_8 \simeq (1)(4.773)(-0.815417554327) \simeq -3.891987986801,\\
u_2 &= (2)(2.06)b_7 \simeq (2)(2.06)(-0.414459473243) \simeq -1.707573029760,\\
u_3 &= (3)(-2.964)b_6 \simeq (3)(-2.964)(0.169804349683) \simeq -1.509900277379,\\
s &= (-3.891987986801) + (-1.707573029760) + (-1.509900277379) \simeq -7.109461293940,\\
b_9 &= s/9 \simeq (-7.109461293940)/9 \simeq -0.789940143771.
\end{aligned}
$$

Coeficiente b_{10}:

$$
\begin{aligned}
u_1 &= (1)(4.773)b_9 \simeq (1)(4.773)(-0.789940143771) \simeq -3.770384306220,\\
u_2 &= (2)(2.06)b_8 \simeq (2)(2.06)(-0.815417554327) \simeq -3.359520323826,\\
u_3 &= (3)(-2.964)b_7 \simeq (3)(-2.964)(-0.414459473243) \simeq 3.685373636074,\\
s &= (-3.770384306220) + (-3.359520323826) + (3.685373636074) \simeq -3.444530993971,\\
b_{10} &= s/10 \simeq (-3.444530993971)/10 \simeq -0.344453099397.
\end{aligned}
$$

Coeficiente b_{11}:

$$
\begin{aligned}
u_1 &= (1)(4.773)b_{10} \simeq (1)(4.773)(-0.344453099397) \simeq -1.644074643423,\\
u_2 &= (2)(2.06)b_9 \simeq (2)(2.06)(-0.789940143771) \simeq -3.254553392337,\\
u_3 &= (3)(-2.964)b_8 \simeq (3)(-2.964)(-0.815417554327) \simeq 7.250692893073,\\
s &= (-1.644074643423) + (-3.254553392337) + (7.250692893073) \simeq 2.352064857313,\\
b_{11} &= s/11 \simeq (2.352064857313)/11 \simeq 0.213824077938.
\end{aligned}
$$

Coeficiente b_{12}:

$$
\begin{aligned}
u_1 &= (1)(4.773)b_{11} \simeq (1)(4.773)(0.213824077938) \simeq 1.020582323996,\\
u_2 &= (2)(2.06)b_{10} \simeq (2)(2.06)(-0.344453099397) \simeq -1.419146769516,\\
u_3 &= (3)(-2.964)b_9 \simeq (3)(-2.964)(-0.789940143771) \simeq 7.024147758413,\\
s &= (1.020582323996) + (-1.419146769516) + (7.024147758413) \simeq 6.625583312893,\\
b_{12} &= s/12 \simeq (6.625583312893)/12 \simeq 0.552131942741.
\end{aligned}
$$

31 Extensión: construir y evaluar las aproximaciones infinitas truncadas

1. Del coeficiente al término.

Una vez conocido b_n, el término evaluado en x es $b_n x^n$. Por ejemplo, para $x = 0.1$, $x^2 = 0.01$ y el término cuadrático es $b_2(0.01)$; no es simplemente b_2. El polinomio hasta grado m suma desde $n = 0$ hasta $n = m$.

2. Cuenta de cada término en el extremo $x = 0.3872$.

n	b_n	x^n	$b_n x^n$	Q_n
0	0.0208896805	1.0000000000	0.0208896805	0.0208896805
1	0.0997064448	0.3872000000	0.0386063354	0.0594960159
2	0.2809821723	0.1499238400	0.0421259262	0.1016219421
3	0.5220558075	0.0580505108	0.0303056063	0.1319275484
4	0.6907073029	0.0224771578	0.0155251370	0.1474526855
5	0.5898244815	0.0087031555	0.0051333342	0.1525860197
6	0.1698043497	0.0033698618	0.0005722172	0.1531582369
7	-0.4144594732	0.0013048105	-0.0005407911	0.1526174458
8	-0.8154175543	0.0005052226	-0.0004119674	0.1522054784
9	-0.7899401438	0.0001956222	-0.0001545298	0.1520509486
10	-0.3444530994	0.0000757449	-0.0000260906	0.1520248580
11	0.2138240779	0.0000293284	0.0000062711	0.1520311291
12	0.5521319427	0.0000113560	0.0000062700	0.1520373991

La última columna es la suma acumulada $Q_n = Q_{n-1} + b_n x^n$, con $Q_0 = b_0$. Por ejemplo, la fila 5 incluye exactamente los términos 0, 1, 2, 3, 4 y 5: son seis términos, no cinco.

3. Error respecto de la misma exponencial de referencia.

Calculamos $e_m(x) = 100|Q_m(x)/e^{F_3(x)} - 1|$. Se evalúa en los 10001 puntos y se toma el máximo. No se usa $100|e^{Q_m - F_3} - 1|$, porque Q_m ya aproxima el coeficiente, no su logaritmo.

Grado	Error relativo máximo (%)
3	13.228276
5	0.359266
8	0.108976
12	0.001574

4. Convergencia y límite físico.

La serie de la exponencial converge para cualquier argumento finito; componerla con un polinomio produce una función entera y su Taylor alrededor de cero tiene radio infinito. Esto es una propiedad matemática. La fracción molar física está entre 0 y 1 y el modelo se estudia aquí solo hasta 0.3872. Convergencia matemática no autoriza extrapolar parámetros ni despreciar oxígeno fuera de sus condiciones.

32 f) Interpretar los resultados físicos con sus signos

1. Tendencia del aluminio.

Para saber si F_3 crece examinamos $F_3' = 4.773 + 4.12x - 8.892x^2 = 4.773 + x(4.12 - 8.892x)$. En el intervalo,

$$8.892x \le 8.892(0.3872) = 3.4429824, \quad 4.12 - 8.892x \ge 0.6770176 > 0.$$

Como $x \ge 0$, el segundo término no es negativo y $F_3' \ge 4.773 > 0$. Así el logaritmo aumenta. La derivada de $\gamma = e^{F_3}$ es $\gamma' = F_3' e^{F_3} > 0$, pues ambos factores son positivos. Por tanto el coeficiente también aumenta.

En cero, $\gamma = e^{-3.8685} \simeq 0.02088968$; en el extremo es 0.152040. Todos esos valores son menores que 1: la solución presenta desviación negativa respecto de la idealidad en esta escala. La actividad $a = x\gamma$ también aumenta porque $a' = \gamma + x\gamma' > 0$.

2. Tendencia del hierro.

Gibbs-Duhem da $H_3' = -xF_3'/(1-x)$. Para $x > 0$, el numerador xF_3' es positivo y el denominador $1 - x$ es positivo; el signo exterior lo hace negativo. Por ello H_3 disminuye desde cero y $\gamma_{\mathrm{Fe}} = e^{H_3}$ disminuye desde 1. En el extremo, $H_3 = -0.529883$ y $\gamma_{\mathrm{Fe}} \simeq 0.58867$.

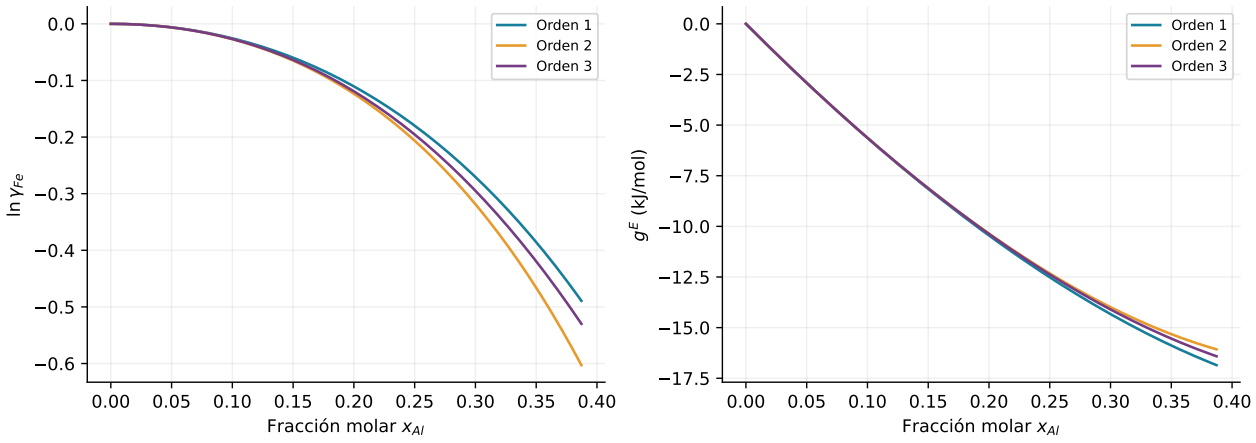


Figura 3. A la izquierda, H_N; a la derecha, $g_N^E/1000$ en kJ/mol. Cada par usa el mismo orden para Al y Fe. El error máximo de energía frente al cúbico es 437.60 J/mol para orden 1 y 341.29 J/mol para orden 2. La energía y el coeficiente del Al no tienen por qué clasificar igual la precisión de un truncamiento: combinan las funciones de forma diferente.

33 f) Energía negativa, calor y estabilidad: tres conceptos distintos

1. Qué dice el signo de g^E.

$g^E = g_{\text{real}} - g_{\text{ideal}}$. Si $g^E < 0$, al pasar de la referencia ideal al modelo real disminuye la energía de Gibbs de mezcla. Pero $g^E = h^E - Ts^E$: contiene entalpía y entropía. Por eso -16.41466 kJ/mol no se interpreta automáticamente como calor liberado.

2. Escribir la energía total de mezcla del modelo.

Sin las contribuciones lineales de los estados puros, que no cambian la segunda derivada,

$$g_{\text{mezcla}} = RT[x\ln x + (1-x)\ln(1-x)] + g^E.$$

El primer corchete es la contribución ideal; el segundo término es la corrección de exceso. Para estabilidad local interesa si la curva se dobla hacia arriba, es decir, si $g''_{\text{mezcla}} > 0$.

3. Derivar la parte ideal dos veces.

Por producto y cadena:

$$(x\ln x)' = \ln x + 1, \qquad [(1-x)\ln(1-x)]' = -\ln(1-x) - 1.$$

Su suma es $\ln x - \ln(1-x)$. Derivando otra vez: $1/x + 1/(1-x)$.

4. Derivar la parte de exceso con Gibbs-Duhem.

Definimos $f_N = xP_N + (1-x)H_N$. Por producto:

$$f'_N = P_N + xP'_N - H_N + (1-x)H'_N.$$

Gibbs-Duhem dice $xP'_N + (1-x)H'_N = 0$; por tanto $f'_N = P_N - H_N$. Derivamos otra vez y sustituimos $H'_N = -xP'_N/(1-x)$:

$$f''_N = P'_N - H'_N = P'_N + \frac{xP'_N}{1-x} = P'_N\frac{1-x+x}{1-x} = \frac{P'_N}{1-x}.$$

5. Sumar ambas contribuciones para la referencia.

$$g''_{\text{mezcla}} = RT\left[\frac{1}{x} + \frac{1}{1-x} + \frac{F'_3}{1-x}\right] > 0 \quad \text{para } 0 < x \le 0.3872.$$

Todos los factores son positivos en ese intervalo. Esto indica estabilidad local frente a pequeñas fluctuaciones de composición dentro del modelo líquido; no compara globalmente su energía con la de sólidos u otras fases.

34 g) Conclusiones, límites y cómo justificar cada afirmación

1. Qué permite conocer Taylor.

La constante describe el límite diluido, la pendiente positiva el aumento inicial de $\ln \gamma_{\mathrm{Al}}$ y las potencias superiores corrigen la forma de esa variación. La expansión permite cuantificar qué se pierde al omitir términos. Esto se sustenta en las derivadas y en la resta explícita de los polinomios.

2. Calidad obtenida.

Para el modelo de referencia, los máximos errores relativos en γ_{Al} son $12.7839\,\%$ con orden 1 y $18.7751\,\%$ con orden 2. El RMSE del logaritmo sí mejora de orden 1 a 2. La compensación entre el término cuadrático positivo y el cúbico negativo explica esta aparente contradicción. El orden 3 tiene error cero solo porque coincide con la referencia.

3. Condiciones de uso.

Con tolerancia de $1\,\%$ en el coeficiente, el lineal funciona hasta $x \simeq 0.07389$ y el cuadrático hasta $x \simeq 0.14973$ en comparación con F_3. Para cubrir la malla completa con esa referencia conviene conservar los cuatro términos. Esos umbrales no certifican exactitud experimental.

4. Dos fuentes diferentes de error.

Una es simplificar Fe-Al-O como Fe-Al; otra es truncar Taylor dentro del binario. Reducir la segunda no elimina la primera. Además están el error del modelo físico frente a mediciones y los efectos de redondeo numérico. No deben llamarse a todos "error de Taylor".

5. Lo que no se puede inferir.

Los coeficientes efectivos no identifican de manera única mecanismos atómicos. No se calcula cinética, no se estima γ_{O} con el binario, no se cambia de temperatura conservando parámetros sin justificación y no se presenta un ajuste experimental independiente. El PDF del artículo no aporta una tabla completa de mediciones EMF con incertidumbres para realizar aquí esa validación.

6. Por qué no extrapolar a aluminio puro con el redondeo.

En $x = 1$, $F_3(1) = -3.8685+4.773+2.06-2.964 = 0.0005$. El H_3 redondeado contiene $0.001\ln(1-x)$, que diverge al aproximarse a 1. Con $c_2 = 2.0595$, $F_{\mathrm{RK}}(1) = 0$ y el logaritmo del hierro se cancela. Un cambio diminuto dentro de la malla puede tener consecuencias grandes al extrapolar a una frontera singular. El programa excluye $x \geq 1$.

Qué acabamos de hacer. Se ha desarrollado Taylor de un modelo declarado, se ha medido su truncamiento y se ha mantenido la consistencia entre especies. Ninguna de esas operaciones convierte automáticamente el modelo en una medición de la naturaleza.

35 e) El código: qué archivo abrir y cómo ejecutarlo

1. Ya no se necesita buscar un archivo que no está adjunto.

El programa didáctico completo está en

`ejemplos/paso_a_paso.py`

y se reproduce íntegro al final de este documento. Incluye importaciones, parámetros, todas las funciones matemáticas, cálculo de métricas, escritura de tablas y generación de cuatro imágenes. No es solo el núcleo de unas veinte líneas. No depende de importar las funciones internas del paquete `feal`.

2. Preparar el entorno.

Descargar o clonar el repositorio y abrir una terminal en su carpeta. `uv` administra el intérprete de Python y sus bibliotecas. `uv.lock` fija las versiones. La primera ejecución necesita conexión para descargar lo que falte. Ejecutar:

```
uv sync --locked
uv run --locked python ejemplos/paso_a_paso.py
```

La primera línea instala el entorno declarado sin modificar el archivo de versiones. La segunda ejecuta el archivo con ese entorno. `python` es el intérprete; la ruta que le sigue es el programa que debe leer.

3. Encontrar los resultados.

Aparecen en `output/didactico/`: `metricas.csv`, `coeficientes_gamma.csv`, `actividad.png`, `errores.png`, `termodinamica.png` y `serie_infinita.png`. La terminal también muestra las métricas. El programa guarda las figuras; no necesita abrir ventanas.

4. Regenerar el documento y los desgloses completos.

El comando integral es:

```
uv run --locked feal
```

Este comando usa los módulos de `src/feal/`, genera tablas y figuras con formato editorial y compila LaTeX. Requiere `pdflatex`; el README indica cómo instalarlo. Con `uv run -locked feal -solo-datos` se omite la compilación. El programa didáctico no necesita LaTeX.

5. Python y MATLAB son lenguajes diferentes.

El archivo entregado es Python. Pegar su contenido en MATLAB no lo ejecuta. Para trasladar el método, hay que traducir sintaxis: Python cuenta índices desde 0 y MATLAB desde 1; NumPy aplica `x**k` elemento a elemento, mientras que en MATLAB se usa `x.^k`. Los CSV sí pueden abrirse desde MATLAB. Esta edición incluye también una versión completa en MATLAB, explicada y listada en las secciones posteriores. Su compatibilidad se comprueba con GNU Octave.

36 e) Leer Python desde cero: signos, arreglos e índices

1. Asignación y comparación.

En código, `x = 0.1` guarda el número 0.1 con el nombre `x`. No es una ecuación que deba resolver el programa. `==` compara igualdad; `<` compara menor que y `<=` menor o igual. El resultado de una comparación es verdadero o falso. `**` eleva a una potencia y `*` multiplica.

2. Comentarios y nombres.

Lo que aparece después de `#` explica la línea y no se ejecuta. Los nombres `np` y `plt` son abreviaturas definidas en las importaciones: representan NumPy y la interfaz de gráficas. Las cadenas entre comillas son texto, no nombres de variables. `f"Orden {orden}"` inserta el valor de `orden` en ese texto.

3. Un arreglo almacena varios números.

`C = np.array([-3.8685, 4.773, 2.06, -2.964])` crea cuatro posiciones. `C[0]` vale -3.8685; `C[1]` vale 4.773; `C[2]` vale 2.06; `C[3]` vale -2.964. La primera posición es 0. `len(C)` sería 4.

4. Operaciones sobre arreglos.

Si `x` contiene $[0, 0.1, 0.2]$, su cuadrado por componentes es $[0, 0.01, 0.04]$. Entonces `2.06*x**2` produce $[0, 0.0206, 0.0824]$. Son tres cálculos independientes, uno por composición, realizados por una sola instrucción. Esta es la razón de que pocas líneas puedan generar muchos puntos.

5. Matrices e índices dobles.

`p` tiene 4 filas y 10001 columnas. `p[2]` es toda la fila del orden 2. `p[2, 0]` sería su primer valor, en $x = 0$. En general, `p[N,j]` corresponde a $P_N(x_j)$. `np.zeros((4, len(x)))` reserva esa matriz llena de ceros; no calcula todavía Taylor.

6. Funciones, ciclos y sangría.

`def` define una función y `return` devuelve su resultado. `for` repite un bloque y `if` elige qué bloque ejecutar. Los espacios al inicio de cada línea indican a qué bloque pertenece: copiarlos mal puede cambiar el programa o causar un error. Definir una función no la ejecuta; hay que llamarla por su nombre y darle los argumentos.

37 e) Seguir un ciclo a mano y entender las funciones matemáticas

1. Seguir `calcular_logaritmo` con $x = 0.1$ y orden 2.

En el programa, x es un arreglo, pero podemos seguir una sola de sus entradas. Inicialmente `resultado=0`. `range(orden+1)` es `range(3)`, que produce $k = 0, 1, 2$; no incluye 3.

k	Potencia	Coeficiente	Término	Resultado acumulado
0	1	-3.8685	-3.8685	-3.8685
1	0.1	4.773	0.4773	-3.3912
2	0.01	2.06	0.0206	-3.3706

Al terminar, `return resultado` entrega -3.3706 para esa entrada. Las otras 10000 entradas recorren las mismas operaciones.

2. Seguir `calcular_diferencia`.

Para orden 1, `range(orden+1,4)` produce $k = 2, 3$. Se inicia $\Delta = 0$; primero se resta $2.06x^2$, luego se resta $-2.964x^3$. El segundo signo negativo produce una suma: $\Delta = -2.06x^2 + 2.964x^3$. Para orden 3, `range(4,4)` está vacío y el resultado sigue en cero.

3. Seguir `calcular_hierro`.

El ciclo exterior selecciona $k = 1, 2, 3$ según el orden. El interior construye el corchete $\ln(1 - x) + \sum_{j=1}^{k} x^j/j$. `pequeno = x < 0.1` es una máscara: una lista de verdaderos y falsos. `x[pequeno]` selecciona solo los puntos verdaderos. En esos lugares se reemplaza el corchete por la cola estable explicada antes. Finalmente se multiplica por kc_k y se suma al hierro acumulado. El programa didáctico espera que x sea un arreglo, como la malla que crea; no se diseñó como interfaz para escalares arbitrarios.

4. Seguir `calcular_coeficientes_gamma` para $n = 2$.

$\min(3, 2) = 2$, así que `range(1,3)` produce $k = 1, 2$. Para $k = 1$, el producto es $1c_1b_{2-1} = c_1b_1$. Para $k = 2$, es $2c_2b_{2-2} = 2c_2b_0$. Se suman y dividen entre $n = 2$. b_0 y b_1 ya existen porque se calcularon antes. Nunca se busca b_{-1}.

38 e) Del cálculo a los archivos y a cada gráfica

1. Organizar resultados.

`calcular()` reúne las funciones anteriores. `metricas.append(...)` añade una fila por orden. El diccionario devuelto asocia nombres de texto con arreglos: `resultados["gamma"]` recupera los coeficientes y `resultados["g"]` las energías. En el diccionario `q`, la clave 5 corresponde a todo el polinomio Q_5 evaluado en la malla.

2. Guardar una tabla.

`guardar_tabla` abre el archivo en modo `"w"`, que lo crea o sustituye en la carpeta de resultados. Escribe primero la cabecera y después las filas. `with` asegura que el archivo se cierre. `zip(range(13), b)` empareja el índice 0 con b_0, el 1 con b_1, y así hasta 12.

3. Dibujar dos paneles.

`plt.subplots(1,2,...)` crea una figura y dos ejes. `ejes[0].plot(x,valores)` dibuja los pares $(x_j, \text{valores}_j)$ en el panel izquierdo. `ejes[1]` identifica el derecho. La etiqueta `label` nombra la curva; `legend()` muestra esos nombres. `set_xlabel` y `set_ylabel` escriben qué significa cada eje. `grid` solo añade líneas de guía.

4. Qué se pasa a cada figura.

Archivo	Panel izquierdo	Panel derecho
`actividad.png`	P_N	e^{P_N}
`errores.png`	$\lvert P_N - F_3\rvert$	$100\lvert e^{P_N - F_3} - 1\rvert$
`termodinamica.png`	H_N	$g_N^E/1000$
`serie_infinita.png`	Q_m y e^{F_3}	$100\lvert Q_m/e^{F_3} - 1\rvert$

Las figuras didácticas usan escalas lineales y muestran todos los órdenes, incluido el 0. Las figuras editoriales del PDF ajustan escalas y selección de curvas para destacar regiones de interés. Los cálculos son equivalentes; las presentaciones no son idénticas.

5. Guardar y ejecutar.

`savefig` escribe la imagen; `dpi=180` indica resolución. `close` libera memoria. `main()` coordina cálculo, tablas y figuras. La condición `if __name__ == "__main__"` lo llama cuando se ejecuta el archivo; no lo llama al importarlo para pruebas. Cada instrucción del listado tiene un comentario inmediato que explica su finalidad.

39 e) Código didáctico completo, con comentarios instrucción por instrucción

Cómo copiarlo. Preferir el archivo del repositorio, que conserva espacios, acentos y sangría. Los números del margen del listado son guías de lectura; no forman parte del código. Las líneas largas pueden partirse visualmente en el PDF sin representar instrucciones nuevas. El archivo completo se ejecuta con el comando indicado; no hay que agregar bloques que falten.

```
"""Programa completo y comentado: Taylor de Fe-Al, tablas y cuatro figuras.

Ejecutar desde la raíz: uv run --locked python ejemplos/paso_a_paso.py
No requiere importar feal ni instalar LaTeX para generar sus resultados.
"""

# Path permite construir rutas sin depender de las barras de cada sistema.
from pathlib import Path
# csv escribe tablas que se pueden abrir en Excel, MATLAB o un editor.
import csv
# NumPy trabaja con muchos números a la vez; lo llamaremos np.
import numpy as np
# Matplotlib dibuja las gráficas; primero elegimos un modo sin ventana.
import matplotlib
# Agg permite guardar imágenes incluso en una computadora sin escritorio.
matplotlib.use("Agg")
# pyplot contiene las instrucciones para crear y guardar figuras.
import matplotlib.pyplot as plt

# Cada posición guarda un coeficiente: C[0], C[1], C[2], C[3].
C = np.array([-3.8685, 4.773, 2.06, -2.964])
# La constante R convierte la energía adimensional a energía por mol.
R = 8.314462618
# Todos los cálculos corresponden a esta temperatura en kelvin.
T = 1873.0
# El extremo procede de la tabla 2; no significa 38.72 % en masa.
X_MAX = 0.3872


# def define una función; no ejecuta todavía las líneas de su interior.
def calcular_logaritmo(x, orden):
    """Sumar, sin ocultar las potencias, P_N = suma de c_k*x**k."""
    # Creamos un arreglo de ceros con una entrada por composición.
    resultado = np.zeros_like(x, dtype=float)
    # range termina antes de su límite: orden+1 incluye el propio orden.
    for k in range(orden + 1):
        # ** es potencia; el cálculo se hace para todas las entradas de x.
        potencia = x**k
        # C[k] selecciona el coeficiente asociado a esa potencia.
        termino = C[k] * potencia
        # Acumulamos el nuevo término sin borrar los anteriores.
        resultado = resultado + termino
    # return entrega el arreglo calculado a quien llamó a la función.
    return resultado


def calcular_diferencia(x, orden):
    """Calcular P_N-F_3 como el negativo de los términos omitidos."""
    # Al principio todavía no hemos sumado ninguna diferencia.
    delta = np.zeros_like(x, dtype=float)
    # Los omitidos comienzan en orden+1 y terminan en k=3.
    for k in range(orden + 1, 4):
        # Restamos cada término ausente: delta = menos el resto de Taylor.
        delta = delta - C[k] * x**k
    # Si orden=3, el ciclo no se ejecuta y devuelve ceros.
```

```
    return delta


def calcular_hierro(x, orden):
    """Integrar Gibbs-Duhem con la condición H_N(0)=0."""
    # Habrá un resultado H por cada composición.
    h = np.zeros_like(x, dtype=float)
    # El término constante no contribuye porque su derivada es cero.
    for k in range(1, orden + 1):
        # log1p(-x) significa logaritmo natural de 1-x.
        corchete = np.log1p(-x)
        # Añadimos x, x**2/2, ... hasta x**k/k.
        for j in range(1, k + 1):
            # / divide; esta línea añade un solo término del corchete.
            corchete = corchete + x**j / j
        # Marcamos las posiciones en las que conviene evitar cancelación.
        pequeno = x < 0.1
        # La cola estable solo se evaluará en las composiciones marcadas.
        cola = np.zeros_like(x[pequeno])
        # range(k+1,33) incluye las potencias desde k+1 hasta 32.
        for j in range(k + 1, 33):
            # La cola tiene signo negativo: -suma de x**j/j.
            cola = cola - x[pequeno]**j / j
        # Sustituimos únicamente las posiciones pequeñas por la cola.
        corchete[pequeno] = cola
        # k*C[k] procede de derivar c_k*x**k antes de integrar.
        h = h + k * C[k] * corchete
    # Para orden 0, el ciclo queda vacío y h permanece igual a cero.
    return h


def calcular_coeficientes_gamma(grado):
    """Obtener b_0 hasta b_grado mediante la recurrencia deducida."""
    # Necesitamos grado+1 espacios porque empezamos a contar desde cero.
    b = np.zeros(grado + 1)
    # np.exp calcula e elevado al argumento; aquí da gamma en cero.
    b[0] = np.exp(C[0])
    # Comenzamos en 1 porque b[0] ya está calculado.
    for n in range(1, grado + 1):
        # Reiniciamos la suma para cada coeficiente nuevo.
        suma = 0.0
        # min impide usar índices negativos cuando n vale 1 o 2.
        for k in range(1, min(3, n) + 1):
            # b[n-k] ya existe porque su índice es menor que n.
            producto = k * C[k] * b[n-k]
            # Incorporamos uno de los productos de la recurrencia.
            suma = suma + producto
        # La división entre n viene del término n*b_n de la derivada.
        b[n] = suma / n
    # Entregamos todos los coeficientes, no solo el último.
    return b


def calcular():
    """Calcular todos los números antes de guardarlos o dibujarlos."""
    # La malla incluye ambos extremos y contiene 10001 números.
    x = np.linspace(0.0, X_MAX, 10001)
    # Una matriz de 4 filas almacena los órdenes 0, 1, 2 y 3.
    p = np.zeros((4, len(x)))
    # h tendrá el logaritmo del hierro, con la misma organización.
    h = np.zeros_like(p)
    # g guardará la energía de exceso en J/mol.
    g = np.zeros_like(p)
    # gamma guarda el coeficiente del Al, distinto de su logaritmo.
    gamma = np.zeros_like(p)
```

```
    # delta guarda el error con signo frente a F_3.
    delta = np.zeros_like(p)
    # error_gamma guarda porcentajes, no fracciones.
    error_gamma = np.zeros_like(p)
    # Las métricas tienen columnas: orden, máximo, RMSE, máximo porcentual.
    metricas = []
    # Ejecutamos exactamente cuatro iteraciones: orden=0,1,2,3.
    for orden in range(4):
        # Una fila completa recibe P_N para todos los x.
        p[orden] = calcular_logaritmo(x, orden)
        # Esta fila recibe H_N compatible con el mismo orden.
        h[orden] = calcular_hierro(x, orden)
        # exp se aplica a cada elemento y recupera el coeficiente.
        gamma[orden] = np.exp(p[orden])
        # Multiplicamos cada especie por su fracción y después por RT.
        g[orden] = R * T * (x*p[orden] + (1-x)*h[orden])
        # Calculamos la diferencia mediante los términos omitidos.
        delta[orden] = calcular_diferencia(x, orden)
        # expm1(z) evalúa exp(z)-1 con precisión para z pequeño.
        error_gamma[orden] = 100 * np.abs(np.expm1(delta[orden]))
        # abs elimina el signo y max elige el mayor valor de la fila.
        maximo = np.max(np.abs(delta[orden]))
        # Primero elevamos al cuadrado cada error.
        cuadrados = delta[orden]**2
        # Sumamos todos los cuadrados.
        suma = np.sum(cuadrados)
        # len(x) es el número de puntos, 10001, no el número de intervalos.
        promedio = suma / len(x)
        # sqrt calcula la raíz cuadrada del promedio.
        rmse = np.sqrt(promedio)
        # Este máximo corresponde a porcentajes del coeficiente.
        maximo_gamma = np.max(error_gamma[orden])
        # append añade una fila al final de la lista de métricas.
        metricas.append([orden, maximo, rmse, maximo_gamma])
    # Para la extensión infinita calculamos 13 coeficientes, del 0 al 12.
    b = calcular_coeficientes_gamma(12)
    # Un diccionario relacionará cada grado con su arreglo de valores.
    q = {}
    # Estos son los cuatro grados que compararemos.
    for grado in (3, 5, 8, 12):
        # Cada polinomio comienza con una suma de ceros.
        aproximacion = np.zeros_like(x)
        # Se incluyen todos los términos desde n=0 hasta n=grado.
        for n in range(grado + 1):
            # Es el Taylor directo de gamma, no la exponencial de P_N.
            aproximacion = aproximacion + b[n] * x**n
        # Guardamos el arreglo bajo su grado: q[3], q[5], q[8] o q[12].
        q[grado] = aproximacion
    # Entregamos un diccionario para poder verificar cada resultado.
    return {"x": x, "p": p, "h": h, "g": g, "gamma": gamma,
            "delta": delta, "error_gamma": error_gamma,
            "metricas": metricas, "b": b, "q": q}


def guardar_tabla(ruta, cabecera, filas):
    """Escribir una tabla CSV con nombres de columnas en español."""
    # with cierra el archivo incluso si ocurre un error durante la escritura.
    with ruta.open("w", encoding="utf-8", newline="") as archivo:
        # writer prepara un escritor que separa las columnas mediante comas.
        escritor = csv.writer(archivo)
        # La primera fila contiene los nombres de las columnas.
        escritor.writerow(cabecera)
        # Después escribimos una fila de datos por cada elemento de filas.
        escritor.writerows(filas)

```

```
186
187 def graficar_pares(x, izquierda, derecha, nombre, etiquetas, salida):
188     """Dibujar dos paneles; cada entrada de la lista será una curva."""
189     # subplots crea una figura con una fila y dos columnas de ejes.
190     figura, ejes = plt.subplots(1, 2, figsize=(10, 4), layout="constrained")
191     # enumerate entrega el número de orden y su arreglo de valores.
192     for orden, valores in enumerate(izquierda):
193         # ejes[0] es el panel izquierdo; plot une pares de puntos.
194         ejes[0].plot(x, valores, label=f"Orden {orden}")
195     # Repetimos para el panel derecho, ejes[1].
196     for orden, valores in enumerate(derecha):
197         # La etiqueta se usará en la leyenda de colores.
198         ejes[1].plot(x, valores, label=f"Orden {orden}")
199     # Asignamos un texto diferente a cada eje vertical.
200     for indice in range(2):
201         # Ambos paneles comparten el significado de x.
202         ejes[indice].set_xlabel("Fracción molar de Al")
203         # etiquetas[0] y etiquetas[1] describen magnitudes diferentes.
204         ejes[indice].set_ylabel(etiquetas[indice])
205         # La cuadrícula ayuda a leer valores sin modificar los datos.
206         ejes[indice].grid(alpha=0.2)
207         # Mostramos la relación entre colores y órdenes.
208         ejes[indice].legend()
209     # / combina una carpeta con un nombre de archivo usando Path.
210     figura.savefig(salida / nombre, dpi=180)
211     # Cerramos esta figura para liberar memoria antes de crear otra.
212     plt.close(figura)
213
214
215 def main():
216     """Ejecutar cálculos, guardar tablas y producir todas las gráficas."""
217     # __file__ es este archivo; parents[1] es la raíz del repositorio.
218     raiz = Path(__file__).resolve().parents[1]
219     # Los resultados didácticos no sustituyen los de la edición del PDF.
220     salida = raiz / "output" / "didactico"
221     # parents crea carpetas intermedias; exist_ok permite repetir el comando.
222     salida.mkdir(parents=True, exist_ok=True)
223     # Ahora sí ejecutamos las operaciones definidas en calcular().
224     resultados = calcular()
225     # Extraemos la malla del diccionario usando su clave de texto.
226     x = resultados["x"]
227     # Guardamos las métricas de todos los órdenes en un archivo.
228     guardar_tabla(salida / "metricas.csv",
229                   ["orden", "error_maximo", "rmse", "error_gamma_porcentaje"],
230                   resultados["metricas"])
231     # zip forma parejas (índice,coeficiente): una por cada b_n.
232     guardar_tabla(salida / "coeficientes_gamma.csv", ["n", "b_n"],
233                   zip(range(13), resultados["b"]))
234     # El primer panel muestra logaritmos y el segundo sus exponenciales.
235     graficar_pares(x, resultados["p"], resultados["gamma"], "actividad.png",
236                    ["ln gamma del Al", "gamma del Al"], salida)
237     # El error absoluto se obtiene tomando abs de la matriz de diferencias.
238     errores = np.abs(resultados["delta"])
239     # Aquí se ven el error del logaritmo y el porcentaje del coeficiente.
240     graficar_pares(x, errores, resultados["error_gamma"], "errores.png",
241                    ["Error absoluto de ln gamma", "Error de gamma (%)"], salida)
242     # Dividir entre 1000 convierte las energías de J/mol a kJ/mol.
243     graficar_pares(x, resultados["h"], resultados["g"] / 1000,
244                    "termodinamica.png", ["ln gamma del Fe", "gE (kJ/mol)"], salida)
245     # Para la serie infinita usamos una figura adicional con dos paneles.
246     figura, ejes = plt.subplots(1, 2, figsize=(10, 4), layout="constrained")
247     # La fila 3 de gamma es exp(F_3), nuestra referencia.
248     referencia = resultados["gamma"][3]
249     # Dibujamos primero esa referencia en el panel izquierdo.
250     ejes[0].plot(x, referencia, color="black", label="Referencia exp(F3)")
```

```
    # items entrega cada pareja grado,arreglo del diccionario q.
    for grado, valores in resultados["q"].items():
        # Mostramos el polinomio directo Q_m en el primer panel.
        ejes[0].plot(x, valores, label=f"Grado {grado}")
        # El error directo usa cociente; valores ya aproxima gamma.
        error = 100 * np.abs(valores / referencia - 1)
        # El segundo panel muestra su error relativo porcentual.
        ejes[1].plot(x, error, label=f"Grado {grado}")
    # Añadimos etiquetas horizontales, cuadrícula y leyenda a ambos paneles.
    for eje in ejes:
        # eje representa uno de los paneles en cada iteración.
        eje.set_xlabel("Fracción molar de Al")
        # Una cuadrícula tenue no oculta las curvas.
        eje.grid(alpha=0.2)
        # legend utiliza los textos label de las llamadas anteriores.
        eje.legend(fontsize=8)
    # El primer eje vertical es el valor del coeficiente.
    ejes[0].set_ylabel("Taylor directo de gamma")
    # El segundo eje vertical es su error porcentual.
    ejes[1].set_ylabel("Error relativo (%)")
    # Guardamos la cuarta imagen; no hace falta abrir una ventana.
    figura.savefig(salida / "serie_infinita.png", dpi=180)
    # Liberamos también esta figura.
    plt.close(figura)
    # Imprimimos una tabla pequeña en la terminal para comprobar la ejecución.
    for fila in resultados["metricas"]:
        # print no modifica los cálculos: solo muestra una fila.
        print(fila)
    # Indicamos dónde quedaron todos los archivos generados.
    print(f"Tablas y cuatro gráficas guardadas en: {salida}")


# Esta condición es cierta al ejecutar el archivo y falsa al importarlo.
if __name__ == "__main__":
    # Llamamos al procedimiento que realiza el trabajo completo.
    main()
```

40 Versión MATLAB: archivo completo y forma de ejecución

1. Qué archivo usar.

La versión MATLAB completa se encuentra en `matlab/paso_a_paso.m`. Incluye una función principal y todas sus funciones auxiliares en el mismo archivo. No requiere copiar funciones desde otro archivo ni instalar un toolbox científico adicional. Los parámetros y las ecuaciones son los mismos que en Python.

2. Abrir la carpeta correcta.

En MATLAB, seleccionar la raíz del repositorio como carpeta actual. En la ventana de comandos ejecutar:

```
addpath('matlab');
resultados = paso_a_paso;
```

`addpath` permite que MATLAB encuentre el archivo dentro de la subcarpeta. La segunda línea llama a la función, realiza el cálculo completo y guarda en `resultados` una estructura con las magnitudes calculadas.

También se puede abrir el archivo y ejecutarlo desde su propia carpeta. El programa localiza su ruta mediante `mfilename('fullpath')`; la carpeta de salida no depende de dónde se abra la consola. No se deben pegar los comentarios del PDF como si fueran órdenes distintas: el listado ya está completo en el archivo descargable.

3. Resultados que se generan.

La carpeta `output/matlab/` contiene cuatro imágenes. Sus nombres son `actividad.png`, `errores.png`, `termodinamica.png` y `serie_infinita.png`. También se guardan `metricas.csv`, `coeficientes_gamma.csv`, `taylor_gamma.csv` y `malla.csv`. La malla contiene los 10001 valores de x y todas las columnas de los polinomios, coeficientes, hierro, energía y expansión directa.

4. Consultar valores en la consola.

```
resultados.metricas
resultados.b
resultados.P(4, end)
resultados.G(4, end)/1000
```

La primera orden muestra las métricas. La segunda, b_0 hasta b_{12}. La tercera toma el orden 3 en la última composición; debe dar aproximadamente -1.883613. La cuarta devuelve la energía del mismo punto en kJ/mol, aproximadamente -16.414661. `end` significa la última posición de esa dimensión.

5. Elegir otra carpeta de salida.

`resultados = paso_a_paso('mis_resultados');` pasa un argumento: el nombre de la carpeta. Si no se da argumento, se usa la ruta predeterminada. Cambiar la carpeta no cambia parámetros ni cálculos.

41 MATLAB desde cero: las diferencias que pueden cambiar una cuenta

1. Índices que comienzan en uno.

Matemáticamente usamos c_0, c_1, c_2, c_3. En MATLAB se guardan en `C(1)`, `C(2)`, `C(3)`, `C(4)`. Por eso la instrucción para c_k es `C(k+1)`. El contador matemático puede valer cero aunque las posiciones del arreglo no.

Para la recurrencia, b_{n-k} se encuentra en `b(n-k+1)`. Ejemplo: con $n = 2$, $k = 2$, se necesita b_0 y la posición es $2 - 2 + 1 = 1$. El resultado b_2 se guarda en `b(3)`.

2. Potencias, productos y divisiones por componentes.

`x.^2` eleva cada entrada de x al cuadrado. `x.*P` multiplica las entradas correspondientes de dos arreglos. `Q./referencia` divide cada entrada de Q entre la correspondiente de la referencia. El punto importa: `*`, `/` y `^` sin punto pueden significar operaciones matriciales diferentes. Multiplicar por un escalar, como `2.06*x.^2`, sí usa `*` porque el factor es un solo número.

3. Filas y columnas.

`zeros(4,M)` crea cuatro filas y M columnas. `P(fila,:)` selecciona todas las columnas de una fila. Para orden 2, `fila=2+1=3`. `P(3,:)` representa P_2 en toda la malla. `P.'` intercambia filas y columnas sin conjugación compleja: se usa para guardar una composición por fila en CSV y para dibujar una curva por columna.

4. Ciclos y cierres.

`for k=0:orden` recorre todos los enteros entre esos límites, incluidos ambos. `end` termina el bloque. Para orden 3, `(orden+1):3` es `4:3`, una secuencia vacía con incremento positivo predeterminado; la suma de términos omitidos queda en cero. El punto y coma al final de una instrucción evita imprimir automáticamente su resultado, pero no evita que se calcule.

5. Funciones y estructuras.

`function p=logaritmo_aluminio(x,orden,C)` define una función que recibe tres argumentos y devuelve `p`. `resultados.P` accede al campo `P` de una estructura. Las llaves de `{'Orden 0','Orden 1'}` crean una colección de textos. `...` al final de una línea indica que la instrucción continúa en la siguiente.

42 MATLAB: seguir las cuentas y comprobar la equivalencia

1. Seguir el orden 2 en una composición.

El ciclo empieza con $p = 0$. Para $k = 0$ usa `C(1)` y $x^0 = 1$; para $k = 1$ usa `C(2)` y x; para $k = 2$ usa `C(3)` y x^2. En $x = 0.1$ acumula -3.8685, luego -3.3912 y finalmente -3.3706. Es la misma cuenta desarrollada antes.

2. Seguir la primera iteración de la recurrencia.

`b(1)=exp(C(1))` fija b_0. Con $n = 1$, el ciclo interior solo tiene $k = 1$; calcula `1*C(2)*b(1)` y divide entre 1. Guarda el resultado en `b(2)`, que representa b_1. Con $n = 2$ ya se pueden usar b_1 y b_0. Así nunca se necesita un coeficiente que aún no exista.

3. Conservar las mismas convenciones físicas.

G está en J/mol y solo se divide entre 1000 al dibujarlo en kJ/mol. P y H son logaritmos; `Gamma=exp(P)` obtiene el coeficiente. El error de la serie infinita usa `Q./referencia-1`, no otra exponencial. La cola del logaritmo se estabiliza con el mismo umbral $x < 0.1$ y hasta grado 32.

4. Comparar automáticamente los archivos.

Después de ejecutar la versión MATLAB o compatible, desde la raíz del repositorio:

```
uv run --locked python pruebas/verificar_matlab.py
```

El verificador compara todas las columnas de los 10001 puntos, los 13 coeficientes de la recurrencia y ambas tablas de errores con el modelo Python. Permite diferencias pequeñas de redondeo entre motores. Si una magnitud no coincide dentro de las tolerancias indicadas en el verificador, termina con error.

5. Entorno de validación disponible.

La ejecución de compatibilidad se realiza con GNU Octave, que interpreta este archivo de sintaxis MATLAB, usando `Dockerfile.octave`. No se dispone de una licencia ni de un motor MATLAB instalado en el entorno de preparación; por tanto la comprobación con Octave no se presenta como una ejecución nativa en MATLAB. El archivo utiliza funciones básicas y evita toolboxes adicionales.

```
docker build -f Dockerfile.octave -t rey-octave .
docker run --rm -v "$PWD:/proyecto" rey-octave
```

En Linux se puede añadir `-user "$(id -u):$(id -g)"` a `docker run` para que los archivos pertenezcan al usuario actual. La receta fija la distribución base declarada, pero no archiva todas las versiones del sistema por digest. El README registra la versión del motor efectivamente comprobada.

43 Código MATLAB completo, comentado instrucción por instrucción

Los comentarios comienzan con `%`; explican la instrucción siguiente y no se ejecutan. Copiar preferentemente `matlab/paso_a_paso.m` desde el repositorio para conservar el archivo completo. Las funciones auxiliares están incluidas después del `end` de la principal.

```
function resultados = paso_a_paso(carpeta_salida)
% SOLUCIONARIO FE-AL: programa completo, comentado y sin toolboxes adicionales.
% Desde la raiz del repositorio: addpath('matlab'); paso_a_paso
% Devuelve resultados y guarda CSV y cuatro figuras PNG.
% Los comentarios explican cada instruccion; el PDF desarrolla las ecuaciones.

% nargin cuenta los argumentos recibidos; cero significa usar la ruta habitual.
if nargin == 0
    % mfilename localiza este archivo; fileparts extrae la carpeta que lo contiene.
    carpeta_codigo = fileparts(mfilename('fullpath'));
    % La carpeta superior a matlab es la raiz del repositorio.
    raiz = fileparts(carpeta_codigo);
    % fullfile une nombres con el separador apropiado para cada sistema.
    carpeta_salida = fullfile(raiz, 'output', 'matlab');
% end cierra el bloque if.
end
% exist(...,'dir') comprueba si ya existe esa carpeta.
if ~exist(carpeta_salida, 'dir')
    % mkdir crea la carpeta; ~ significa negacion logica.
    mkdir(carpeta_salida);
end

% MATLAB cuenta desde 1: C(1)=c_0, C(2)=c_1, C(3)=c_2, C(4)=c_3.
C = [-3.8685, 4.773, 2.06, -2.964];
% Constante de los gases en J/(mol K).
R = 8.314462618;
% Temperatura fija en kelvin.
T = 1873;
% El limite de composicion procede de la tabla 2 del articulo.
x_max = 0.3872;
% linspace incluye ambos extremos; x es un vector fila de 10001 posiciones.
x = linspace(0, x_max, 10001);
% numel cuenta todas las posiciones del vector.
M = numel(x);
% Cada fila de P representa un orden y cada columna una composicion.
P = zeros(4, M);
% H almacenara los logaritmos del coeficiente del hierro.
H = zeros(4, M);
% G almacenara energia de exceso en J/mol.
G = zeros(4, M);
% Gamma almacenara el coeficiente del aluminio, no su logaritmo.
Gamma = zeros(4, M);
% Delta guardara P_N-F_3 con su signo.
Delta = zeros(4, M);
% Error_gamma guardara el error relativo porcentual del coeficiente.
Error_gamma = zeros(4, M);
% Columnas: orden, error maximo, RMSE y maximo error porcentual.
metricas = zeros(4, 4);

% 0:3 produce 0,1,2,3; aqui el orden matematico sigue empezando en cero.
for orden = 0:3
    % La fila fisica del arreglo es orden+1 porque MATLAB empieza en uno.
    fila = orden + 1;
    % Dos puntos selecciona todas las columnas de esa fila.
    P(fila, :) = logaritmo_aluminio(x, orden, C);
    % Integramos Gibbs-Duhem para el mismo orden, no siempre para el cubico.
    H(fila, :) = logaritmo_hierro(x, orden, C);
```

```
    % exp se aplica a cada componente del vector.
    Gamma(fila, :) = exp(P(fila, :));
    % .* multiplica elemento por elemento; RT convierte a J/mol.
    G(fila, :) = R*T*(x.*P(fila, :) + (1-x).*H(fila, :));
    % Calculamos los terminos omitidos para evitar restar numeros casi iguales.
    Delta(fila, :) = diferencia_logaritmo(x, orden, C);
    % expm1(z)=exp(z)-1; abs elimina el signo de cada resultado.
    Error_gamma(fila, :) = 100*abs(expm1(Delta(fila, :)));
    % max escoge el mayor error absoluto entre todas las composiciones.
    error_maximo = max(abs(Delta(fila, :)));
    % .^2 eleva cada error al cuadrado; ^2 seria una potencia matricial.
    cuadrados = Delta(fila, :).^2;
    % sum suma los 10001 cuadrados del vector.
    suma = sum(cuadrados);
    % Dividimos entre el numero de puntos, no entre el de intervalos.
    promedio = suma/M;
    % sqrt obtiene la raiz cuadrada del promedio.
    rmse = sqrt(promedio);
    % Este maximo corresponde al error de gamma en porcentaje.
    error_porcentual = max(Error_gamma(fila, :));
    % Los corchetes forman una fila de cuatro numeros.
    metricas(fila, :) = [orden, error_maximo, rmse, error_porcentual];
end

% Calculamos b_0,...,b_12; el resultado tendra 13 posiciones.
b = coeficientes_exponencial(12, C);
% Estos grados incluyen, respectivamente, 4,6,9 y 13 terminos.
grados = [3, 5, 8, 12];
% Cada fila de Q representa uno de esos cuatro grados.
Q = zeros(4, M);
% Guardaremos el grado y su error relativo maximo.
metricas_infinita = zeros(4, 2);
% La fila 4 corresponde al orden 3; es exp(F_3), la referencia.
referencia = Gamma(4, :);
% indice indica la posicion dentro de la lista grados.
for indice = 1:numel(grados)
    % Recuperamos el grado concreto: 3,5,8 o 12.
    grado = grados(indice);
    % Recorremos todos los terminos, incluido el constante n=0.
    for n = 0:grado
        % b(n+1) representa b_n; .^ aplica la potencia a cada composicion.
        Q(indice, :) = Q(indice, :) + b(n+1)*x.^n;
    end
    % ./ divide entrada por entrada; Q ya aproxima gamma directamente.
    error = 100*abs(Q(indice, :)./referencia - 1);
    % Una fila guarda el grado y el mayor porcentaje obtenido.
    metricas_infinita(indice, :) = [grado, max(error)];
end

% Construimos la ruta del primer CSV y escribimos sus columnas.
escribir_csv(fullfile(carpeta_salida, 'metricas.csv'), ...
    'orden,error_maximo,rmse,error_gamma_porcentaje', metricas);
% (0:12).' convierte un vector fila a columna sin conjugacion compleja.
escribir_csv(fullfile(carpeta_salida, 'coeficientes_gamma.csv'), ...
    'n,b_n', [(0:12).', b.']);
% Guardamos tambien los errores de la expansion directa de gamma.
escribir_csv(fullfile(carpeta_salida, 'taylor_gamma.csv'), ...
    'grado,error_max_porcentaje', metricas_infinita);
% Transponemos cada matriz para que una fila del CSV sea una composicion.
datos = [x.', P.', H.', G.', Gamma.', Q.'];
% La cabecera enumera las columnas en el mismo orden que datos.
cabecera = ['x,P0,P1,P2,P3,H0,H1,H2,H3,G0,G1,G2,G3,' ...
    'gamma0,gamma1,gamma2,gamma3,Q3,Q5,Q8,Q12'];
% Escribimos las 10001 filas calculadas para compararlas con Python.
escribir_csv(fullfile(carpeta_salida, 'malla.csv'), cabecera, datos);
```

```
123
124 % Las llaves crean una lista de textos, llamada arreglo de celdas.
125 leyendas = {'Orden 0', 'Orden 1', 'Orden 2', 'Orden 3'};
126 % Primera figura: logaritmos a la izquierda, coeficientes a la derecha.
127 graficar_par(x, P, Gamma, 'ln gamma del Al', 'gamma del Al', ...
128     leyendas, fullfile(carpeta_salida, 'actividad.png'));
129 % Segunda figura: error absoluto y error relativo porcentual.
130 graficar_par(x, abs(Delta), Error_gamma, 'Error absoluto de ln gamma', ...
131     'Error de gamma (%)', leyendas, fullfile(carpeta_salida, 'errores.png'));
132 % Tercera figura: dividir G entre 1000 convierte J/mol a kJ/mol.
133 graficar_par(x, H, G/1000, 'ln gamma del Fe', 'gE (kJ/mol)', ...
134     leyendas, fullfile(carpeta_salida, 'termodinamica.png'));
135 % La cuarta figura necesita leyendas por grado, no por orden del logaritmo.
136 leyendas_q = {'Grado 3', 'Grado 5', 'Grado 8', 'Grado 12'};
137 % Creamos una matriz para los porcentajes de cada Q.
138 errores_q = zeros(size(Q));
139 % Recorremos las cuatro filas ya calculadas.
140 for indice = 1:4
141     % El cociente se calcula por componentes, por eso se usa ./.
142     errores_q(indice, :) = 100*abs(Q(indice, :)./referencia - 1);
143 end
144 % Dibujamos los Q y sus errores; la referencia se usa en el calculo del error.
145 graficar_par(x, Q, errores_q, 'Taylor directo de gamma', ...
146     'Error relativo (%)', leyendas_q, ...
147     fullfile(carpeta_salida, 'serie_infinita.png'));
148
149 % struct crea un contenedor con campos identificados por nombre.
150 resultados = struct();
151 % El punto accede a un campo del contenedor, no multiplica.
152 resultados.x = x;
153 % Guardamos cada magnitud para poder consultarla en la consola.
154 resultados.P = P;
155 resultados.H = H;
156 resultados.G = G;
157 resultados.Gamma = Gamma;
158 resultados.Q = Q;
159 resultados.b = b;
160 resultados.metricas = metricas;
161 % disp muestra valores sin alterar el calculo.
162 disp('Columnas: orden, error maximo, RMSE, error de gamma (%)');
163 disp(metricas);
164 % fprintf inserta la ruta en %s y termina con un salto de linea \n.
165 fprintf('Tablas y cuatro figuras guardadas en: %s\n', carpeta_salida);
166 % Fin de la funcion principal; las funciones auxiliares siguen en el mismo archivo.
167 end
168
169 function p = logaritmo_aluminio(x, orden, C)
170 % Implementa P_N=sumatoria de c_k*x^k.
171 % size(x) conserva el numero de posiciones y la orientacion de x.
172 p = zeros(size(x));
173 % Incluimos el termino k=0, que es la constante.
174 for k = 0:orden
175     % .^ significa potencia por elemento.
176     potencia = x.^k;
177     % La posicion k+1 guarda el coeficiente matematico c_k.
178     termino = C(k+1)*potencia;
179     % Acumulamos los terminos sin sustituir la suma anterior.
180     p = p + termino;
181 end
182 end
183
184 function delta = diferencia_logaritmo(x, orden, C)
185 % Implementa el negativo de la suma de terminos omitidos.
186 % La diferencia comienza en cero para cada composicion.
187 delta = zeros(size(x));
```

```
% (orden+1):3 es vacio si orden=3; no se ejecuta ninguna iteracion.
for k = (orden+1):3
    % Restar el termino cubico negativo produce una contribucion positiva.
    delta = delta - C(k+1)*x.^k;
end
end

function h = logaritmo_hierro(x, orden, C)
% Implementa la integral analitica de Gibbs-Duhem, con estabilizacion diluida.
% h empieza en cero, compatible con H_0=0.
h = zeros(size(x));
% No se incluye k=0 porque la derivada de una constante es cero.
for k = 1:orden
    % log1p(-x) calcula el logaritmo natural de 1-x.
    corchete = log1p(-x);
    % Añadimos los primeros k terminos positivos.
    for j = 1:k
        % Cada potencia se divide por su indice j.
        corchete = corchete + x.^j/j;
    end
    % La mascara logica marca donde x es menor que 0.1.
    pequeno = x < 0.1;
    % Seleccionamos unicamente esas posiciones para la cola estable.
    xs = x(pequeno);
    % Reservamos un cero para cada posicion seleccionada.
    cola = zeros(size(xs));
    % Sumamos hasta j=32; el PDF acota el resto omitido.
    for j = (k+1):32
        % Todos los terminos de esta cola son negativos o cero.
        cola = cola - xs.^j/j;
    end
    % Reemplazamos solo los puntos pequeños, conservando el resto.
    corchete(pequeno) = cola;
    % El factor k procede de derivar c_k*x^k.
    h = h + k*C(k+1)*corchete;
end
end

function b = coeficientes_exponencial(grado, C)
% Implementa la recurrencia para el Taylor directo de exp(F_3).
% El vector tiene grado+1 posiciones porque incluye b_0.
b = zeros(1, grado+1);
% La posicion 1 corresponde a b_0, no a b_1.
b(1) = exp(C(1));
% El contador n conserva el indice matematico del coeficiente buscado.
for n = 1:grado
    % Reiniciamos la suma para este nuevo coeficiente.
    suma = 0;
    % min evita pedir coeficientes de indice matematico negativo.
    for k = 1:min(3, n)
        % C(k+1)=c_k y b(n-k+1)=b_(n-k).
        producto = k*C(k+1)*b(n-k+1);
        % Incorporamos un producto a la suma.
        suma = suma + producto;
    end
    % La posicion n+1 corresponde a b_n; dividimos entre el n matematico.
    b(n+1) = suma/n;
end
end

function escribir_csv(ruta, cabecera, datos)
% Guarda numeros con 17 cifras significativas, utiles para comparaciones.
% fopen abre el archivo para escritura; -1 indica que no pudo abrirlo.
archivo = fopen(ruta, 'w');
% Detectamos un error de permisos o de ruta antes de escribir.
```

```
if archivo == -1
    % error interrumpe la ejecucion con un mensaje legible.
    error('No se pudo abrir el archivo: %s', ruta);
end
% onCleanup garantiza el cierre al salir, incluso si ocurre un error.
cerrar = onCleanup(@() fclose(archivo)); %#ok<NASGU>
% Escribimos los nombres de columnas y un salto de linea.
fprintf(archivo, '%s\n', cabecera);
% size(datos,1) cuenta filas; recorremos una por composicion o metrica.
for fila = 1:size(datos, 1)
    % El primer numero se escribe sin coma delante.
    fprintf(archivo, '%.17g', datos(fila, 1));
    % Las columnas siguientes si llevan una coma antes del numero.
    for columna = 2:size(datos, 2)
        % %.17g conserva 17 cifras significativas de cada numero.
        fprintf(archivo, ',%.17g', datos(fila, columna));
    end
    % Terminamos la fila despues de escribir todas sus columnas.
    fprintf(archivo, '\n');
end
end

function graficar_par(x, izquierda, derecha, titulo_izq, titulo_der, etiquetas, ruta)
% Dibuja y guarda dos paneles; no requiere interaccion con ventanas.
% Para curvas decrecientes, la leyenda abajo evita ocultar el inicio.
if strcmp(titulo_izq, 'ln gamma del Fe')
    % southwest significa esquina inferior izquierda.
    posicion_leyenda = 'southwest';
else
    % Para las curvas crecientes dejamos libre la zona inferior.
    posicion_leyenda = 'northwest';
end
% figure crea una figura invisible de tamaño suficiente para leer las etiquetas.
figura = figure('Visible', 'off', 'Position', [100, 100, 1100, 440]);
% Primer panel: una fila, dos columnas, posicion 1.
subplot(1, 2, 1);
% .' transpone sin conjugar; cada columna pasa a representar una curva.
plot(x, izquierda.', 'LineWidth', 1.2);
% xlabel indica el significado del eje horizontal.
xlabel('Fraccion molar de Al');
% ylabel indica la magnitud calculada del panel izquierdo.
ylabel(titulo_izq);
% legend asocia un texto con cada curva en el mismo orden.
legend(etiquetas, 'Location', posicion_leyenda);
% grid on agrega lineas de referencia visual.
grid on;
% Activamos el segundo panel de la misma figura.
subplot(1, 2, 2);
% Dibujamos los datos correspondientes al panel derecho.
plot(x, derecha.', 'LineWidth', 1.2);
% Ambos paneles tienen la misma variable horizontal.
xlabel('Fraccion molar de Al');
% El eje vertical cambia segun la propiedad.
ylabel(titulo_der);
% Esta leyenda usa los mismos ordenes o grados que la primera.
legend(etiquetas, 'Location', posicion_leyenda);
% La cuadricula no modifica las curvas.
grid on;
% print exporta PNG; -r150 establece la resolucion en puntos por pulgada.
print(figura, ruta, '-dpng', '-r150');
% Cerramos la figura para liberar memoria.
close(figura);
end
```

44 Cómo funciona la reproducción editorial y cómo se comprueba

1. Ubicación de cada responsabilidad.

`src/feal/modelo.py` contiene las funciones matemáticas reutilizables. `reproducir.py` evalúa la malla, escribe tablas, produce figuras y llama a LaTeX. `desglose.py` escribe los pasos intermedios y los CSV de auditoría. El ejemplo didáctico hace explícitos los ciclos y acumulaciones que el módulo compacto expresa con sumas o funciones de NumPy.

2. Equivalencia con la evaluación compacta.

El módulo reutilizable usa `np.polynomial.polynomial.polyval` para evaluar un polinomio. Por ejemplo,

$$c_0 + c_1x + c_2x^2 + c_3x^3 = c_0 + x[c_1 + x(c_2 + xc_3)].$$

Al distribuir los paréntesis interiores aparecen $c_2x + c_3x^2$; al multiplicar nuevamente por x y sumar c_1x aparecen $c_1x + c_2x^2 + c_3x^3$. La forma anidada reduce operaciones. Puede diferir en los últimos bits por el orden de redondeo, pero representa el mismo polinomio que el ciclo didáctico.

3. Tablas y texto sincronizados.

Los comandos `input` de LaTeX insertan las tablas generadas, incluidas las multiplicaciones de la recurrencia. Las figuras se leen de los PDF vectoriales calculados. LaTeX se ejecuta tres veces para resolver el índice y las referencias a páginas. El PDF final se escribe en `output/pdf/solucionario.pdf`.

4. Verificar en vez de confiar solo en la apariencia.

```
uv run --locked pytest
```

Las pruebas verifican las derivadas mediante álgebra simbólica exacta, Gibbs-Duhem para cada orden, el hierro frente a integración numérica independiente, las métricas de referencia y la recurrencia frente a una expansión simbólica de la exponencial. También comparan el programa didáctico con el módulo reutilizable y verifican la estructura y operaciones de los archivos de desglose.

5. Reproducibilidad.

`uv.lock` fija bibliotecas y `.python-version` el intérprete. Se fijan las fechas internas del PDF y se registran huellas SHA-256 en `output/manifiesto.json`. Las salidas del ejemplo didáctico se guardan aparte y no forman parte de ese manifiesto editorial. Las versiones de TeX Live y las fuentes dependen del sistema: no se promete identidad binaria entre plataformas diferentes. En el mismo entorno se compara una reconstrucción limpia y una segunda ejecución.

45 Guía para responder las dudas que motivaron esta edición

«¿Dónde está Taylor?»

En la construcción $c_k = F^{(k)}(0)/k!$, con las derivadas calculadas término a término y evaluadas en cero. La parte previa deriva la función que se va a desarrollar; esa parte previa no debe confundirse con Taylor.

«¿Por qué sale lo mismo que ya estaba?»

Porque el modelo del logaritmo es un polinomio cúbico. Su Taylor de grado tres lo recupera. Cortar antes sí crea una aproximación con error calculable.

«¿De dónde toma los datos para las gráficas?»

De evaluar las ecuaciones en una malla que elegimos. Solo los parámetros de partida proceden del artículo; los miles de puntos no son miles de observaciones experimentales.

«¿De dónde toma los coeficientes de la serie infinita?»

De $b_0 = e^{c_0}$ y la recurrencia que se deduce al igualar potencias en $\gamma' = F_3'\gamma$. Se muestran todos los coeficientes utilizados, con sus productos, sumas y divisiones.

«¿El fragmento corto era todo el programa?»

No. El fragmento del PDF original era solo un núcleo. Esta edición incorpora un archivo ejecutable completo y su listado comentado; incluye las instrucciones para generar y guardar las gráficas.

«¿Puedo reproducir un número específico?»

Sí: identificar si es P_N, γ_N, H_N, g_N^E o Q_m; localizar su definición; sustituir el x correspondiente; seguir el ejemplo y consultar el CSV. Para P_N y sus errores, `desglose_malla.csv` deja explícitos los términos y sus combinaciones. Para la serie infinita, consultar `coeficientes_gamma.csv` y `terminos_gamma_extremo.csv`.

46 Fuentes y correspondencia con el examen

[1] Ishfaq, M.; Pande, M. M. (2025). *Thermodynamic analysis of Fe-Al-O system using Redlich-Kister (R-K) polynomial and Wagner's Taylor series expansion.* The Journal of Chemical Thermodynamics, 202, 107415. DOI: 10.1016/j.jct.2024.107415. Copia original: `referencias/articulo.pdf`. Parámetros: tabla 1, p. 5. Modelo y relaciones: ecuaciones (16)–(26), p. 6. Composiciones de referencia: tabla 2, p. 7. Reducción binaria: sección 3.2.

[2] Universidad Veracruzana, Facultad de Ciencias Químicas. *Primer Examen Parcial: Matemáticas Aplicadas.* Maestría en Ingeniería Química, 6 de octubre de 2026. Copia: `referencias/parcial.pdf`.

[3] *Solucionario razonado: Series de Taylor y termodinámica del sistema Fe-Al-O.* Documento proporcionado, fechado el 7 de octubre de 2026. Se conserva sin cambios en la carpeta `referencias/`, con el nombre `solucionario_original.pdf`. Esta edición amplía sus explicaciones, reconstruye sus resultados y agrega código íntegro y desgloses; no reproduce su maquetación original.

Inciso	Contenido desarrollado
a	Fenómeno de desoxidación, variables, actividades, unidades y aproximación binaria.
b	Elección de Taylor, función a desarrollar, centro en cero y diferencia con Fourier.
c	Derivación parcial molar, álgebra del modelo, derivadas de Taylor, factoriales y Gibbs-Duhem.
d	Truncamientos, ejemplos, errores, máximos, RMSE y tolerancias.
e	Programa completo, instrucciones de ejecución, gráficas, código comentado y pruebas.
f	Interpretación de tendencias, energía, estabilidad y límites del modelo.
g	Conclusiones fundamentadas, condiciones de uso y límites de las afirmaciones.

Convención final. Se usa punto decimal para coincidir con Python y con las cifras de las ecuaciones. Los valores del texto se redondean solo al imprimir. Los originales se mantienen íntegros y sus huellas constan en `referencias/SHA256SUMS`.